



Measuring Popularity Bias, Fairness for Users, and Feedback Loops in Recommender Systems

by
Cebo Mkhwanazi

Supervisor: Prof M.V. Gwetu
Co-supervisor: Dr S.O Sunday

June 2026



Declaration

I, **Cebo Mkhwanazi**, declare that this document, entitled *Measuring Popularity Bias, Fairness for Users, and Feedback Loops in Recommender Systems*, is my own work, that it has not been submitted before for any degree or examination at this or any other institution, and that all sources I have used or quoted have been indicated and acknowledged by means of complete references.

I further declare that:

1. The work submitted is the result of my own investigation, except where otherwise stated in the text and fully referenced.
2. I have not allowed, and will not allow, anyone else to copy my work with the intention of passing it off as their own work.
3. Where generative artificial intelligence (AI) tools were used in the preparation of this thesis, their use was limited to support tasks (for example, language editing, code debugging, or literature organisation) and did not replace my own analysis, experimental design, interpretation of results, or academic argument. All substantive claims, tables, figures, and conclusions are my responsibility and have been verified against the underlying data and code.
4. I understand that misrepresentation of authorship, plagiarism, or undisclosed use of AI in ways that breach Stellenbosch University's policies on academic integrity may lead to disciplinary action.
5. I grant permission to Stellenbosch University to reproduce this thesis, in whole or in part, for library and archival purposes and for inclusion in the institutional repository, subject to the University's policies on copyright and embargo.
6. Where this thesis draws on proprietary or restricted data (including the Takealot e-commerce corpus), I have complied with the access, anonymisation, and use conditions agreed with the data provider and my supervisors; no confidential material that would breach those agreements is disclosed beyond what is necessary for academic evaluation.

Signature:_____

Date:_____

Name: Cebo Mkhwanazi

Abstract

Recommender Systems (RS) are critical tools for content discovery, yet many algorithms fail to promote diverse content, instead exhibiting a measurable popularity bias the disproportionate recommendation of already popular items. This phenomenon leads to significant exposure disparity, reinforcing market imbalances and restricting user access to niche content. This thesis presents a rigorous empirical study that detects and quantifies inherent popularity and exposure bias across classical Collaborative Filtering (CF) architectures and Content-Based models.

Using standard benchmark datasets (MovieLens, Last.fm, Book-Crossing) alongside a proprietary, highly sparse e-commerce dataset from Takealot, the study implements a unified, three-phase evaluation framework. **Experiment A** measures macro-level exposure bias, revealing that Matrix Factorization models (SVD, ALS) act as extreme popularity amplifiers, exhibiting near-perfect Gini coefficients and collapsing catalog coverage on sparse data. **Experiment B** investigates user-centric fairness, quantifying a "niche tax" where models significantly degrade in ranking quality (NDCG@10) and predictive accuracy (RMSE) for users with non-mainstream tastes. **Experiment C** employs a longitudinal feedback simulation to demonstrate how CF models drive diversity decay over time, while content-based models show resilience against filter bubbles.

The results provide quantitative evidence that architectural differences significantly contribute to bias amplification, independent of the underlying data distribution. The severe exacerbation of these biases in the sparse Takealot environment highlights the danger of relying solely on dense academic benchmarks for model selection. These findings underscore the necessity of adopting holistic evaluation metrics beyond traditional accuracy, and demonstrate that mitigating algorithmic concentration is essential to ensure long-term ecosystem health and fairness in digital platforms.

Acknowledgements

First and foremost, I thank God for the strength, perseverance, and clarity needed to complete this degree. To my parents, siblings, and wider family: thank you for your patience, encouragement, and belief in me throughout long nights of reading, coding, and writing. This work would not have been possible without your support at home.

I am deeply grateful to my supervisor, **Prof. M.V. Gwetu**, for his guidance, intellectual rigour, and steady mentorship from the earliest research questions through to the final manuscript. I thank my co-supervisor, **Dr. S.O. Sunday**, for constructive feedback, methodological advice, and for helping me keep the empirical programme focused and defensible. Their combined supervision shaped both the scope and the quality of this thesis.

This research was conducted within the **School for Data Science and Computational Thinking** at **Stellenbosch University**. I thank the School’s academic and administrative staff for creating an environment in which applied, data-intensive research is taken seriously, and for the resources and collegial culture that made sustained experimental work feasible.

I acknowledge the **Standard Bank Lab for Recommender Systems** for institutional support, collaboration space, and a community of researchers working on recommendation, fairness, and real-world machine learning problems. Conversations with lab members sharpened my thinking about evaluation beyond accuracy and about the gap between benchmark studies and production-like sparsity.

I thank **Stellenbosch University** for the opportunity to pursue postgraduate study and for upholding standards of academic integrity that this work aspires to meet. I am grateful to colleagues and peers who read drafts, debated results, and shared practical advice on tooling and reproducibility.

For data and domain context, I thank those who facilitated access to the proprietary Takealot e-commerce corpus under appropriate confidentiality and use constraints. I also acknowledge the maintainers and research communities behind the public benchmarks used in this study (MovieLens, Last.fm 2K, and Book-Crossing), without whose openly documented datasets comparative recommender-system research would be far poorer.

Any errors or omissions in interpretation remain my own. To everyone named above and to those I have failed to mention by name but who helped along the way: thank you.

Contents

Declaration	1
Abstract	2
Acknowledgements	3
List of Abbreviations	9
1 Introduction	12
1.1 Context and Motivation	12
1.1.1 Technical, Social, and Economic Imperatives	13
1.2 Problem Statement	13
1.3 Research Objectives and Questions	14
1.4 Scope and Limitations	15
1.4.1 Focus Area	15
1.4.2 System Scale and Constraints	16
1.4.3 Methodological Limitations	16
1.5 Key Contributions	16
1.6 Thesis Outline	17
2 Literature Review	19
2.1 Recommender Systems Paradigms	19
2.1.1 Memory-Based Collaborative Filtering	20
2.1.2 Model-Based Collaborative Filtering	20
2.1.3 Content-Based and Hybrid Systems	21
2.1.4 Summary of the Six Baseline Families	22
2.1.5 Scope Boundary: Deep Learning and Graph Models	23
2.1.6 Identified Research Gap	23
2.2 Theoretical Foundations of Bias	24
2.3 Popularity and Exposure Bias: The Propagation Chain	26
2.4 User-Centric Fairness and Disparate Impact	28
2.5 Temporal Dynamics and Feedback Loops	30
2.6 Mitigation strategies: a methodological survey	35
2.7 Theoretical framework	39
2.7.1 The Bias Propagation Chain model	40
2.7.2 Constructs and formal definitions	42
2.7.3 Hypotheses	44
2.7.4 The Sparsity Amplification Conjecture	44
2.8 The research gap	45

3	Methodology and Experimental Framework	50
3.1	Research Design Overview	50
3.2	Datasets and Domains	50
3.3	Data Preprocessing	51
3.4	Ethics and Data Use	51
3.5	Algorithmic Baselines & Constraints	51
3.6	Evaluation Metrics Formulations	51
3.6.1	Macro-Level Metrics (Distributional Equity)	51
3.6.2	Micro-Level Metrics (Ranking Quality & Accuracy)	52
3.7	Statistical Testing Framework	52
3.8	Implementation and Reproducibility	52
3.9	Longitudinal feedback simulator (Experiment C)	53
3.10	Threats to validity	53
4	Experiment A: Macro-Level Exposure Bias	54
4.1	Purpose and alignment with the research design	54
4.2	Protocol and computational details	54
4.3	Concentration and catalogue coverage	54
4.4	Popularity amplification and tail behaviour	55
4.5	Average recommendation popularity and accuracy	56
4.6	Post-processing mitigation: popularity-aware re-ranking	56
4.7	Discussion and answer to RQ1	57
5	Experiment B - User-Centric Fairness (The "Niche Tax")	58
5.1	User Stratification Strategy	58
5.2	The RMSE Methodological Artifact	58
5.3	Predictive Disparities	58
5.4	Ranking Fairness (NDCG@10)	58
5.5	Summary of Findings	60
6	Experiment C - Longitudinal Feedback Dynamics	61
6.1	Simulation Design	61
6.2	The Stochastic User Choice Model	61
6.3	Collaborative "Popularity Creep"	61
6.4	Content Model Resilience	61
6.5	Summary of Findings	61
7	Discussion	63
7.1	The Interconnectedness of Biases	63
7.2	The Domain Extreme	63
7.3	Threats to Validity	63
7.4	Implications for Practitioners	63
8	Conclusion	64
8.1	Summary of the Study	64
8.2	Key Takeaways	64
8.3	Directions for Future Research	64
	References / Bibliography	64

A	Extended tables for macro and micro metrics	69
A.1	Macro-level (catalog) metrics	69
A.2	Micro-level (user-centric) metrics	71
A.3	Longitudinal feedback simulation (full iteration grid)	72
B	Additional visualizations and diagrams	74
B.1	End-to-end evaluation pipeline	74
B.2	Stylised feedback loop with explicit simulator knobs	74
B.3	Cross-dataset figures (same assets as the main text)	74
C	Extended software specifications	78
C.1	Repository layout	78
C.2	Language runtime and hardware assumptions	78
C.3	Core dependency pins	78
C.4	Pinned versions (<code>pip freeze</code> excerpt)	78
C.5	Regeneration checklist	79

List of Figures

2.1	The <i>Bias Propagation Chain</i> : MNAR training data and exposure-coupled objectives produce macro-level concentration, uneven segment-level service, and feedback-generated logs that re-enter training. Macro diagnostics, user-centric tests, and longitudinal simulation (Experiments A–C) interrogate successive links of the same loop.	41
2.2	Relations among constructs used in this thesis: training-side popularity skew and exposure-coupled missingness shape segment-level disparities; feedback and retraining link user-level outcomes to longitudinal diversity loss, with catalog starvation as the extreme macro outcome.	43
3.1	The three main steps of the testing plan.	50
4.1	Macro-level exposure inequality (left) and catalogue coverage (right). Darker Gini cells indicate stronger concentration; coverage is read directly from the colour scale. Heatmaps are produced from the merged results table after <code>run_all_datasets.py</code> (public corpora) and the Takealot pipeline.	55
5.1	Fairness gap $\Delta\text{RMSE} = \text{RMSE}_{\text{niche}} - \text{RMSE}_{\text{mainstream}}$ by dataset and model. Positive values indicate higher error for niche users. Note that content models that fall back to a global mean for rating prediction can show flat ΔRMSE ; segment NDCG should be read alongside this chart.	59
5.2	RMSE by user segment (Niche vs. Mainstream) across datasets and models.	59
6.1	Feedback simulation: aggregate diversity (unique recommended items) and ARP over five iterations for SVD and TF-IDF, by dataset. Trajectories illustrate how collaborative vs. content-based models respond to synthetic clicks.	62
B.1	Unified pipeline: shared preprocessing and model training feed three evaluation tracks (Chapters 4–6).	75
B.2	Longitudinal simulator: logs evolve from recommendations and stochastic clicks, then re-enter training.	75
B.3	Appendix: Gini heatmap	75
B.4	Appendix: catalog coverage heatmap	76
B.5	Appendix: niche RMSE gap	76
B.6	Appendix: RMSE by segment	77
B.7	Appendix: feedback diversity	77
C.1	Code and artefact layout relative to the research repository root.	78

List of Tables

2.1	Illustrative coverage of key dimensions in selected prior work (Y = addressed; (Y) = partial; empty = not central). The final row sketches this thesis.	46
4.1	Spearman ρ between training popularity and recommendation-slot frequency (higher implies stronger popularity alignment). Values come from <code>combined_macro_bias_metrics.csv</code> .	
4.2	Aggregate diversity (unique recommended items) and long-tail share of recommendation slots (fraction from the least popular 20% of training items).	56
4.3	Test RMSE for models that expose global rating-scale predictors in the macro pipeline (empty cells where RMSE is not computed in this export).	56
A.1	Macro metrics (part I): concentration and popularity alignment.	69
A.2	Macro metrics (part II): tail mass, diversity, metadata entropy, RMSE.	70
A.3	Micro metrics (part I): RMSE by taste segment and Δ RMSE.	71
A.4	Micro metrics (part II): NDCG@10 by taste segment.	71
A.5	Feedback simulation: aggregate diversity and ARP at every iteration (SVD vs. TF-IDF).	72
C.1	Scientific stack directly exercised by <code>run_all.py</code>	79

List of Abbreviations

The following abbreviations and acronyms are used throughout this thesis. Model names in code (e.g. `LogReg`, `RandomForest`) follow the implementation in `run_all.py`. Each table spans the full text width with abbreviations in the first column and definitions in the second.

General abbreviations and metrics

Abbrev.	Meaning
AI	Artificial intelligence (including generative tools declared in the Declaration).
ALS	Alternating least squares; implicit matrix factorisation baseline (<code>implicit</code> library).
ARP	Average recommendation popularity: mean training-set interaction count of recommended items, averaged over users.
CF	Collaborative filtering: recommendations from user–item interaction patterns.
Δ GAP	Absolute difference in group average popularity between two user segments.
Δ RMSE	Difference in root mean squared error between niche and mainstream user segments (niche minus mainstream).
GAP	Group average popularity: mean popularity of items recommended to a user group.
Gini	Gini coefficient of the item recommendation-frequency distribution (0 = perfect equality, 1 = maximal concentration).
IPS	Inverse propensity scoring; debiasing estimator discussed in the literature (not implemented in this thesis).
k -core	Iterative filtering that retains users and items with at least k interactions in the bipartite graph.
k -NN	k -nearest neighbours; user-based collaborative filtering baseline (<code>surprise.KNNBasic</code>).
LT@20%	Long-tail share: fraction of recommendation slots assigned to items in the least popular 20% of the training catalogue.
MF	Matrix factorisation; latent-factor collaborative filtering.
ML-100K	MovieLens 100K benchmark dataset.
ML-1M	MovieLens 1M benchmark dataset.
MNAR	Missing not at random; missingness depends on unobserved preference and exposure.
NDCG	Normalized discounted cumulative gain; ranking-quality metric with position discounting.
NDCG@ K	NDCG computed at cutoff K (this thesis uses $K=10$ unless stated otherwise).
NeuMF	Neural matrix factorisation (excluded from experiments; cited for scope).
RF	Random forest regressor on label-encoded user and item identifiers.
RMSE	Root mean squared error between predicted and held-out ratings.
RQ	Research question (RQ1–RQ3 map to Experiments A–C).
RS	Recommender system(s).
RecSys	Recommender-systems research community and conference context.
SKU	Stock keeping unit; individual product identifier in e-commerce catalogues.
SVD	Biased probabilistic matrix factorisation via <code>surprise.SVD</code> (latent-factor baseline).
TF–IDF	Term frequency–inverse document frequency; content-based vectorisation of item metadata.

Baseline shorthand (tables and figures)

Abbrev.	Meaning
LogReg	Per-user logistic regression on TF-IDF features (high vs. low rating vs. user median).
RandomForest	Random forest on encoded <code>user_id</code> and <code>item_id</code> pairs.
SVD+Rerank	Post-processing re-rank on top of SVD scores with a popularity penalty ($\lambda=0.8$ on MovieLens 100K and Takealot).
TF-IDF	Content-based recommender using cosine similarity between user profiles and item vectors.

Chapter 1

Introduction

1.1 Context and Motivation

Every time a user opens a streaming service, browses an online store, or scrolls through a social media feed, a recommender system is working in the background to decide what content to show them. These systems have become the primary gatekeepers of content discovery on modern digital platforms: they determine which films appear on a homepage, which products appear in search results, which articles get surfaced in a news feed, and which songs play next. At scale, the decisions made by these systems shape not just individual user experiences but the broader economics of digital markets determining which products sell, which artists get heard, and which vendors survive.

Recommender systems learn to make these decisions by studying historical user behaviour: the records of what users have clicked on, purchased, rated, or watched in the past. The underlying assumption is that patterns in historical behaviour are informative about future preferences that a user who enjoyed certain films in the past will enjoy similar films in the future, or that a user who purchased certain products is likely to need related ones. This assumption is broadly reasonable, and it has enabled the development of powerful systems that genuinely help users navigate large catalogs they could not feasibly explore on their own.

However, this learning process carries a structural flaw that is the central subject of this thesis. Historical engagement data is not a neutral, representative sample of user preferences across the full catalog. It is a heavily skewed sample in which a small fraction of items the popular ones, the ones that have been heavily promoted, the ones that happened to be surfaced by prior recommendation systems have accumulated the overwhelming majority of interactions. When a recommendation algorithm learns from this data, it learns not just about user preferences but also about which items have historically been visible. It then encodes this visibility pattern into its predictions, recommending popular items more frequently, which makes those items even more visible, which generates even more interactions, which reinforces the algorithm’s confidence in recommending them. The result is a self-reinforcing cycle in which popular items become progressively more dominant and the vast majority of the catalog the “long tail” of niche, specialist, and emerging products becomes progressively less visible.

This phenomenon is called **popularity bias**, and its consequences extend well beyond a technical inconvenience. They are felt by users who receive recommendations that do not reflect their genuine preferences, by producers of niche items who are algorithmically excluded from discovery, and by platforms whose long-term health depends on a diverse and thriving ecosystem of content and products. Understanding, measuring, and ultimately mitigating popularity bias is therefore not merely an academic exercise in algorithmic fairness it is a practical imperative for anyone building or operating a recommender system at scale.

1.1.1 Technical, Social, and Economic Imperatives

The urgency of addressing popularity bias can be understood through three distinct but interconnected lenses: technical, social, and economic. Each lens reveals a different dimension of the problem and a different reason why it demands serious attention.

From a **technical perspective**, the standard approach to building and evaluating recommender systems is fundamentally misaligned with the goal of genuine discovery. Most systems are trained to minimise prediction error on historical interactions to learn to predict, as accurately as possible, what users have already done. But when the historical data is skewed toward popular items, optimising for this objective teaches the algorithm to recommend popular items, not because they are the best match for each individual user, but because they appear frequently enough in the training data that guessing them reduces average error. The result is a system that scores well on standard accuracy benchmarks while simultaneously failing at its core purpose: helping users find items they would not have discovered on their own. A system that only recommends what users could easily find themselves provides little technical value. Furthermore, as popular items increasingly dominate recommendation lists, the system suffers from **catalog starvation** a state in which only a tiny fraction of the available inventory is ever surfaced to users, effectively wasting the majority of the catalog. This is not a minor inefficiency; it represents a fundamental failure of the system’s discovery function [52].

From a **social perspective**, unchecked popularity bias creates real and measurable harm to users. When recommendation engines consistently narrow the range of content shown to users, they create what researchers have called **filter bubbles** [46] information environments in which users are exposed primarily to content that reinforces their existing preferences and behaviour, rather than broadening their horizons. This homogenisation of experience is not just a loss of diversity for its own sake; it actively disadvantages users whose preferences diverge from the mainstream. A user who enjoys niche genres, specialist products, or minority-taste content receives systematically worse recommendations than a user whose preferences align with the popular majority not because their preferences are harder to satisfy, but because the algorithm has less training data for the items they prefer and defaults to popular alternatives instead. This disparity in service quality across user segments is a fairness problem: the system is not serving all users equally, and the users it serves least well are those whose tastes are already least catered to by the broader platform [5].

From an **economic perspective**, popularity bias poses a direct threat to the long-term viability of large digital marketplaces. The economic promise of online retail was closely tied to the concept of the **long tail** [7]: the observation that a digital platform, unconstrained by physical shelf space, can stock enormous catalogs and profitably serve niche demand that physical retailers could never accommodate. The aggregate revenue from many niche products, each selling rarely but together constituting a substantial market, was supposed to be unlocked by recommendation systems that helped users discover items they would never have found through browsing alone [13]. Popularity bias directly undermines this promise. When a recommender concentrates visibility on already-popular items, it creates a winner-takes-all dynamic [7] in which niche vendors are algorithmically suppressed, emerging products never gain traction, and the platform’s ecosystem gradually consolidates around a small number of dominant items and sellers. For a platform like Takealot, which relies on a large ecosystem of third-party vendors, this consolidation reduces long-term resilience and ultimately harms both the vendors who depend on the platform for visibility and the users who would benefit from access to a broader range of products.

1.2 Problem Statement

The existence of popularity bias in recommender systems is not a new discovery. The research community has documented it across a wide range of platforms, algorithms, and datasets, and

has proposed numerous mitigation strategies [37]. What remains significantly underexplored, however, is the *comparative* manifestation of this bias across different algorithmic families and across datasets with fundamentally different sparsity profiles.

Most existing studies examine popularity bias in the context of one or two specific algorithms typically collaborative filtering variants on one or two specific datasets typically dense academic benchmarks like MovieLens. This narrow focus makes it difficult to answer questions that are practically important for system designers: Does the choice of algorithm family (collaborative versus content-based) systematically change the severity of popularity bias, or is the bias primarily a function of the data? How does bias manifest differently when a system is deployed on an ultra-sparse retail catalog versus a dense movie-rating dataset? Can the same metrics and conclusions that hold on academic benchmarks be trusted to transfer to real e-commerce environments?

This thesis addresses these questions through a structured empirical investigation built around four specific gaps in the existing literature:

1. **The Matthew Effect in algorithmic cycles.** Popularity bias does not simply reflect a static property of the training data it creates self-reinforcing cycles in which mainstream items receive more recommendations, generate more interactions, and thereby accumulate stronger training signals that lead to yet more recommendations. This cumulative advantage mechanism [45], well-documented in sociology, operates in recommender systems through the feedback loop between recommendations and interaction logs. Understanding how different algorithmic architectures participate in and amplify this cycle is essential for designing systems that do not progressively concentrate visibility over time.
2. **Incomplete characterisation across algorithmic families.** There is a notable absence of rigorous comparative studies that place memory-based collaborative methods (k -NN), model-based collaborative methods (SVD and ALS), and content-based methods (TF-IDF, logistic regression, random forests) side by side under identical experimental conditions and measure their bias profiles on the same metrics. Without such comparisons, it is impossible to attribute observed bias patterns to specific architectural choices rather than to dataset properties or evaluation artefacts.
3. **Insufficient user-centric analysis.** System-level aggregate statistics a single RMSE or Precision@K score averaged across all users routinely mask the unequal distribution of recommendation quality across different user segments. Users with niche preferences are systematically underserved relative to mainstream users, yet this disparity is invisible in aggregate metrics. A rigorous evaluation of the “niche tax” requires per-user analysis and explicit comparison across mainstreamness cohorts [5].
4. **Poorly understood quality–fairness trade-offs.** The relationship between a model’s predictive accuracy and its fairness measured as the equity of recommendation quality across user segments and the diversity of item exposure across the catalog remains poorly characterised in unmitigated, baseline environments. Before interventions can be designed and evaluated, this baseline relationship must be established empirically.

1.3 Research Objectives and Questions

The overarching objective of this thesis is to establish a rigorous empirical baseline that characterises how popularity bias manifests across three distinct scales macro-level item exposure, user-segment fairness, and longitudinal feedback dynamics for multiple algorithmic families and dataset domains, including an ultra-sparse real-world e-commerce environment.

This objective is operationalised through three research questions, each corresponding to one scale of analysis and one experimental chapter:

- **RQ1:** How concentrated are recommendations across items under different algorithmic paradigms, and how does this concentration vary across datasets with different sparsity profiles?

This question asks about the *macro-level* behaviour of each algorithm: whether it distributes recommendation slots broadly across the catalog or concentrates them on a small number of popular items. The answer is quantified using distributional inequality metrics (Gini coefficient, catalogue coverage, average recommendation popularity) rather than accuracy metrics, because accuracy metrics can remain stable even as concentration becomes severe.

- **RQ2:** How do accuracy and ranking quality differ across user segments defined by their preference for mainstream versus niche items, and how does this disparity vary across algorithmic families and dataset sparsity levels?

This question asks about the *user-centric* behaviour of each algorithm: whether it serves all users equally well or systematically advantages mainstream users at the expense of niche users. The answer is quantified by partitioning users into mainstreamness cohorts and comparing RMSE and NDCG@10 across cohorts for each algorithm on each dataset.

- **RQ3:** Under a stylised longitudinal feedback process in which a model is repeatedly retrained on interaction logs generated by its own prior recommendations, how do diversity and popularity-related metrics evolve over successive rounds, and do collaborative and content-based models exhibit different trajectories?

This question asks about the *dynamic* behaviour of recommendation algorithms over time: whether popularity bias compounds through the feedback loop between recommendations and interaction logs, and whether content-based models whose predictions depend on item features rather than interaction counts are more resilient to this compounding than collaborative models.

1.4 Scope and Limitations

1.4.1 Focus Area

This research focuses specifically on **popularity bias** and **exposure bias** in recommender systems that learn from both explicit feedback (numerical ratings provided directly by users) and implicit feedback (interaction signals such as clicks, purchases, or play counts inferred from user behaviour). The study examines how these biases manifest at three scales: the macro level (catalog coverage and item exposure distribution), the micro level (recommendation quality across user segments), and the longitudinal level (evolution of diversity metrics over simulated feedback rounds).

A deliberate choice was made to focus on *measuring and characterising* bias in unmitigated baseline systems rather than on proposing and evaluating new mitigation algorithms. This choice reflects a specific gap in the literature: rigorous baseline characterisation across multiple algorithms and domains is a prerequisite for effective mitigation design, yet it is less common in the literature than mitigation proposals. By establishing what the bias landscape looks like before any correction is applied, this thesis provides the empirical foundation that mitigation work requires.

The research explicitly excludes several related topics. It does not address **demographic bias** disparities in recommendation quality linked to user attributes such as gender, age, or ethnicity because such analysis requires demographic data that is not available in the datasets used. It does not evaluate **online A/B testing** in live production environments, because all evaluations are conducted offline using historical datasets and simulations. The longitudinal analysis of Experiment C uses a stylised simulation rather than real retraining cycles, for the reasons discussed in Section 3.9.

1.4.2 System Scale and Constraints

The algorithmic scope of this study is intentionally bounded to well-established, interpretable baseline families. Deep learning architectures such as Neural Collaborative Filtering (NeuMF) [31], Neural Graph Collaborative Filtering (NGCF) [54], and LightGCN [32] are excluded for two reasons. First, their computational requirements make large-scale cross-dataset comparative evaluation impractical without specialised hardware. Second, and more fundamentally, their complexity makes it difficult to attribute observed bias patterns to specific architectural choices: when a deep model exhibits high Gini concentration, it is not straightforward to identify which component of the architecture is responsible. The classical and content-based baselines used in this thesis have transparent, well-understood mechanisms that allow bias patterns to be traced back to specific structural properties of the algorithm a prerequisite for the diagnostic, characterisation-oriented goals of this study.

The dataset scope covers five corpora spanning a range of interaction densities: MovieLens 100K, MovieLens 1M, Last.fm 2K, Book-Crossing, and the proprietary Takealot e-commerce dataset. The first four are publicly available academic benchmarks; the fifth is a real-world retail dataset included specifically to stress-test conclusions in an ultra-sparse environment. The generalisability of findings is accordingly bounded: conclusions should be interpreted as applying to systems and environments with sparsity profiles similar to those represented in these five datasets, not as universal claims about all possible recommender system deployments.

1.4.3 Methodological Limitations

Several methodological limitations should be kept in mind when interpreting the results of this thesis.

Offline evaluation as a proxy for online performance. All experiments in this thesis are conducted offline, using historical interaction data. Offline metrics are imperfect proxies for the online user experience: they cannot capture the effects of position bias (users clicking on items simply because they appear high in the list), novelty effects (users being more engaged with items they have not previously encountered), or the complex dynamics of real user decision-making under recommendation. Results should be interpreted as characterising algorithmic behaviour on historical data rather than as predictions of live system performance.

Stylised feedback simulation. The longitudinal feedback simulation in Experiment C uses a stochastic click model that blends predicted relevance with item popularity to generate synthetic interactions. While this model is grounded in theoretical considerations about how users balance personal relevance against social proof, it is not calibrated to real user behaviour data. The simulation captures qualitative trends whether diversity increases or decreases, whether ARP rises or falls more reliably than it captures quantitative magnitudes.

Mean-fallback RMSE for content models. Content-based models in this study are designed to produce ranked recommendation lists rather than explicit rating predictions. When RMSE is computed for these models to enable direct comparison with collaborative filtering models that do produce rating predictions unpredicted items are assigned the global training mean as a fallback value. This introduces a methodological artefact: the RMSE of content models reflects the accuracy of this mean-fallback approximation rather than a genuine predictive signal, making direct RMSE comparisons between content and collaborative models unreliable. NDCG@10, which measures ranking quality rather than rating prediction accuracy, is the more appropriate comparison metric for content-based models, and results are interpreted accordingly throughout the thesis.

1.5 Key Contributions

This thesis makes four specific contributions to the recommender systems research literature.

First, a unified three-phase evaluation framework. This thesis introduces and applies a single, coherent evaluation protocol that simultaneously measures popularity bias at three scales: macro item exposure (Experiment A), user-segment fairness (Experiment B), and longitudinal feedback dynamics (Experiment C) for six algorithmic families across five datasets. The unification of these three measurement scales within a single study, using consistent preprocessing, splitting, and metric definitions, is itself a methodological contribution: it enables the relationships between macro concentration, user-level disparity, and temporal dynamics to be studied directly, rather than inferred by comparing across separate papers with different experimental conditions.

Second, a controlled cross-paradigm comparison of bias profiles. By evaluating memory-based collaborative filtering (k -NN), latent-factor collaborative filtering (SVD, ALS), and multiple content-based pipelines (TF-IDF, logistic regression, random forest) under identical conditions, this thesis provides the first systematic empirical characterisation of how algorithmic architecture independently of dataset properties shapes the severity and form of popularity bias. The finding that ALS exhibits near-perfect Gini concentration and catalog starvation while logistic regression on content features maintains coverage orders of magnitude higher is a quantitative, architecture-attributable result that has not previously been documented under controlled cross-paradigm conditions.

Third, an ultra-sparse e-commerce stress test. The inclusion of the proprietary Takealot corpus with interaction densities orders of magnitude lower than standard academic benchmarks reveals failure modes that are mechanically suppressed by the abundance of interactions in MovieLens-style datasets. The demonstration that matrix factorisation models collapse to recommending only 14 distinct items across the full test cohort on the Takealot dataset, while content-based models maintain coverage of over half the catalog, provides concrete empirical evidence that benchmark-derived conclusions do not transfer to real e-commerce environments.

Fourth, a controlled comparison of feedback loop trajectories. By running longitudinal feedback simulations for both SVD and TF-IDF under identical click-generation conditions, this thesis provides the first direct controlled comparison of how collaborative and content-based models respond to the feedback loop between recommendations and interaction logs. The finding that TF-IDF maintains or increases aggregate diversity over successive rounds while SVD exhibits progressive popularity creep supports the theoretical prediction that content-based models’ structural decoupling from interaction counts confers resilience against feedback-driven concentration.

1.6 Thesis Outline

The remainder of this document is organised as follows. Each empirical chapter answers one research question (RQ1–RQ3) under a shared offline protocol; Chapters 2 and 3 supply the conceptual and operational foundations.

Chapter 2 (Literature Review) Surveys collaborative and content-based recommender paradigms, missing-not-at-random (MNAR) feedback, macro-level popularity and exposure metrics, user-centric fairness, longitudinal feedback dynamics, and post-processing mitigations. The chapter closes with the *Bias Propagation Chain* theoretical framework and testable hypotheses (H1–H3) that link macro concentration, segment-level disparity, and diversity decay over simulated feedback rounds.

Chapter 3 (Methodology) Specifies datasets (MovieLens 100K and 1M, Last.fm 2K, Book-Crossing, and the proprietary Takealot corpus), k -core preprocessing, six baseline families (k -NN, SVD, ALS, TF-IDF, per-user logistic regression, random forest on identifiers), metric definitions (Gini, catalogue coverage, ARP, NDCG@10, RMSE by taste segment), the stochastic click simulator for Experiment C, statistical testing, implementation details, and threats to validity.

Chapter 4 (Experiment A, RQ1) Quantifies catalogue-level concentration and popularity amplification across algorithms and domains, including optional SVD popularity re-ranking on MovieLens 100K and Takealot. Answers whether collaborative latent-factor and implicit models concentrate exposure more strongly than content-based pipelines under matched protocols (H1).

Chapter 5 (Experiment B, RQ2) Segments users into niche, diverse, and mainstream cohorts and reports RMSE and NDCG@10 disparities the “niche tax.” Examines whether accuracy and ranking quality diverge across segments and how sparsity (especially on Takealot) shapes those gaps (H2).

Chapter 6 (Experiment C, RQ3) Runs five-round feedback simulations for SVD versus TF-IDF with a blended relevance–popularity click model. Tracks aggregate diversity and ARP over iterations to test whether collaborative filtering amplifies popularity while content-based models remain more resilient (H3).

Chapter 7 (Discussion) Synthesises findings across the three experiments, relates macro catalog starvation to segment-level unfairness and feedback loops, discusses the Takealot domain extreme, and revisits validity threats and implications for practitioners.

Chapter 8 (Conclusion) Summarises contributions, states key takeaways, and outlines directions for future work (e.g. calibrated click models, stronger debiasing, and live evaluation).

References Bibliography of sources cited in the main text.

Appendices Appendix A provides extended macro- and micro-metric tables exported from the evaluation pipeline; Appendix B repeats principal figures and workflow diagrams; Appendix C documents repository layout, dependencies, and reproducibility steps.

Chapter 2

Literature Review

This chapter reviews the landscape of recommender system research that is directly relevant to this thesis. It examines the traditional and modern algorithmic approaches used to build recommenders, explains why the data these systems learn from is inherently skewed and biased, and traces how algorithms then *amplify* those skews in ways that are unfair to both users and less popular items. It also covers how researchers have attempted to measure fairness from the user’s perspective, how feedback loops cause bias to compound over time, and what kinds of post-processing techniques have been proposed to reduce bias after a model has already been trained.

The goal of this chapter is not to serve as an exhaustive encyclopedia of every paper ever written on recommender systems. Rather, it is to methodically build toward a specific gap in the current body of research: existing work has not produced a unified, rigorous evaluation that simultaneously measures (1) how unevenly items are exposed across an entire catalog at a macro level, (2) how that uneven exposure falls disproportionately on different segments of users, and (3) how these two problems compound each other over time through feedback loops especially in real-world e-commerce settings where product catalogs are extraordinarily sparse (meaning most users have interacted with only a tiny fraction of available items). This chapter lays the conceptual groundwork that makes that gap visible.

2.1 Recommender Systems Paradigms

Recommender systems are generally classified according to two criteria: (1) what type of data signals they use as input, and (2) what mathematical assumptions known as **inductive biases** they bake into the model about how users and items relate to one another [?].

To understand what an inductive bias is, consider a simple analogy. Suppose you are trying to guess what movie a friend will enjoy next. If you assume “my friend will like whatever is most popular right now,” that is an inductive bias. If instead you assume “my friend will like whatever is most similar to movies they have previously rated highly,” that is a different inductive bias. In machine learning, an inductive bias is the set of structural assumptions an algorithm encodes assumptions about the shape of the world that allow it to make predictions on data it has never seen before [?]. These assumptions are not neutral: they determine which signals in the training data the model pays attention to and which it ignores. Because recommender systems are trained on historical user behaviour, and because that historical behaviour is already skewed (popular items have far more interactions than niche ones), the inductive biases of an algorithm directly govern how much it will reproduce, reduce, or amplify those skews when making future recommendations [34].

This section reviews the traditional and contemporary algorithmic paradigms that form the set of baseline models evaluated in this thesis. For each paradigm, we explain both how it works and how its fundamental mathematical structure makes it more or less vulnerable to popularity

bias and data sparsity.

2.1.1 Memory-Based Collaborative Filtering

Collaborative Filtering (CF) is built on a simple but powerful intuition: *people who agreed in the past will probably agree in the future* [6]. If you and another user both rated several films similarly, then a film that person enjoyed but you have not yet seen is a reasonable recommendation for you. Crucially, CF does this using *only* the historical record of who interacted with what it deliberately ignores any description of the items themselves (such as genre, director, or plot) and any demographic information about users (such as age or location). Its only raw material is the matrix of past interactions.

The collaborative framework was first deployed at scale by the GroupLens system, which filtered Usenet news articles using correlation scores between users [47]. As web platforms grew to millions of users, however, computing similarity between every pair of users in real time became computationally impractical. To solve this, researchers shifted the approach toward **item-based k -nearest neighbours (k -NN)** [49]. Rather than asking “which users are most similar to this user?”, item-based k -NN asks “which items are most similar to items this user has already interacted with?” Similarity between items is measured by how often they appear together in the same users’ interaction histories a quantity called **co-occurrence**. Item-to-item similarities tend to be more stable over time than user-to-user similarities, and they can be precomputed offline, making the system much faster to serve at query time.

Today, neighborhood-based methods remain standard academic benchmarks, partly because of accessible open-source libraries such as the Surprise library [?]. Their enduring appeal also stems from their **interpretability**: if a system recommends an item to a user, a human can trace exactly why the system found items with high co-occurrence scores to things the user previously interacted with. There are no hidden layers or abstract embeddings obscuring the logic.

However, this reliance on raw co-occurrence counts is precisely where a significant bias problem enters. Consider an extremely popular item a blockbuster film, a bestselling product, or a viral song. Because it has been interacted with by an enormous number of users across the platform, it will naturally co-occur frequently with almost *every* other item in the catalog. Its co-occurrence score with virtually everything is artificially inflated simply by virtue of its popularity. As a result, k -NN similarity matrices tend to be dominated by these mainstream items, and the recommender ends up surfacing popular items at the top of nearly every recommendation list, regardless of whether those items are actually relevant to a particular user’s specific preferences [39]. Niche items, regardless of how genuinely relevant they might be to specific users, are mathematically crowded out.

2.1.2 Model-Based Collaborative Filtering

Rather than storing an explicit table of similarity scores between items and querying it at prediction time, **model-based collaborative filtering** uses the interaction history to train a compact mathematical model a learned compression of the data that can then generate predictions for unseen user-item pairs.

Matrix Factorization

The most prominent architecture within this class is **Matrix Factorization (MF)**, which achieved widespread adoption following the Netflix Prize competition, where it significantly outperformed prior approaches [38]. The core idea is elegant: take the large, sparse matrix of user-item interactions (where rows are users, columns are items, and cells contain ratings or interaction signals), and decompose it into two much smaller matrices. One matrix assigns each user a short numerical vector (called a **latent vector** or **embedding**) representing their

underlying taste profile. The other matrix assigns each item its own latent vector representing the item’s underlying characteristics. “Latent” here means *hidden* these dimensions are not hand-coded features like “genre: action” or “price: \$20”; they are abstract dimensions that the model discovers purely from the pattern of interactions. The predicted preference of a given user for a given item is then computed as the dot product (a measure of alignment) between their respective latent vectors. If a user’s taste vector and an item’s feature vector point in similar directions in this abstract space, the model predicts a high preference.

Implicit Feedback and Alternating Least Squares

When platforms collect data passively tracking which items users click on, view, add to cart, or purchase, without ever asking them to assign a rating the signal is called **implicit feedback** [33]. This is a fundamentally different type of data from explicit star ratings. With explicit ratings, a missing cell in the interaction matrix simply means “we do not know what this user thinks of this item.” With implicit feedback, a missing cell is *informative*: it might mean the user has never encountered this item, or that they saw it and were not interested, or that they are simply not in the target market for it. These interpretations are very different, and the model must handle this ambiguity carefully. The widely used **Alternating Least Squares (ALS)** algorithm [33] addresses this by assigning *confidence weights* to observations: a user-item pair that resulted in a purchase gets high confidence, whereas an unobserved pair gets low (but non-zero) weight, encoding the uncertainty about what the absence of interaction means.

How Sparsity Amplifies Popularity Bias in Matrix Factorization

Matrix factorization imposes a specific constraint on the solution: it must find a *low-rank* approximation of the full interaction matrix. “Low-rank” means the model assumes that the true underlying structure of user preferences can be captured by a small number of latent dimensions that the world is simpler than the raw data suggests. This is a powerful regularisation strategy (a way of preventing overfitting), but it has a direct consequence for how niche items are treated.

To construct a meaningful latent vector for an item, the model needs enough interaction data to triangulate where that item “sits” in the latent space. Popular items, which have hundreds or thousands of interactions, produce well-defined, stable embeddings. Niche items, which might have only a handful of interactions, produce noisy, poorly defined embeddings that the model cannot reliably position in the latent space. Because the model is trained to minimise prediction error across all users, it prioritises getting predictions right for the large number of users who have interacted with popular items. The result is that the learned low-dimensional space is disproportionately shaped by popular items, and the model systematically assigns higher predicted scores to them [52]. This is not a deliberate design choice it is a mathematical consequence of optimising for accuracy in a world where interaction data is heavily skewed toward popular items.

2.1.3 Content-Based and Hybrid Systems

Content-Based Filtering operates from a fundamentally different premise: instead of asking “what do users similar to this user like?”, it asks “what items are similar to items *this specific user* has liked?” Crucially, similarity here is defined not by co-occurrence in interaction histories but by the intrinsic descriptive properties of the items themselves metadata such as text descriptions, tags, genres, keywords, or structural attributes [42].

The system builds a **taste profile** for each user by aggregating the feature signatures of items they have previously engaged with. Future recommendations are then generated by finding items in the catalog whose features best match this profile. Because this scoring mechanism is entirely decoupled from how many other users have interacted with any given item, content-based methods can theoretically surface niche items without any popularity penalty: if a little-known

item’s metadata closely matches a user’s profile, it can be recommended regardless of its global interaction count.

TF-IDF as a Text Feature Extractor

At the foundational level, this thesis implements content-based pipelines using **Term Frequency–Inverse Document Frequency (TF-IDF)** [?], a classical technique from information retrieval (the field of study concerned with searching and retrieving relevant documents). TF-IDF represents each item as a numerical vector derived from its text description. The “Term Frequency” component scores how often a word appears in a particular item’s description. The “Inverse Document Frequency” component then down-weights words that appear in almost every item’s description (common words like “the” or “product”), ensuring that rarer, more discriminating words carry more weight. The resulting vectors can then be compared using standard vector similarity measures to identify items with similar descriptive content.

Higher-Complexity Content Models

This thesis also implements more complex content models, including per-user logistic regression trained on text features and non-linear tree ensembles (random forests) trained on item identifiers [? ?]. These allow us to test whether increasing the complexity of the content-based pipeline meaningfully changes how bias behaves or whether the fundamental immunity to popularity bias that content methods enjoy is preserved regardless of model complexity.

Hybrid Systems and Why This Thesis Keeps Families Separate

In commercial production environments, it is common to combine collaborative and content-based signals into **hybrid systems**, typically because doing so improves accuracy metrics [6]. However, this thesis deliberately keeps the two families separated and evaluates them side by side under identical experimental conditions. The reason is methodological: if we want to understand *why* a system exhibits a certain bias profile, we need to be able to trace that behaviour back to specific mathematical components. Blending two paradigms before measuring bias makes it impossible to attribute the observed outcome to either one. By keeping them isolated, we can make clean, causal claims about how each architecture’s structural assumptions lead to the bias patterns we observe.

2.1.4 Summary of the Six Baseline Families

The core argument of this section is that an algorithm’s mathematical assumptions are not just abstract technical details they are the primary mechanism through which historical data skews are encoded, amplified, or suppressed. The six baseline families used in this thesis instantiate different structural assumptions, creating a spectrum of bias vulnerabilities:

- **Latent-Factor Models** (Biased MF/SVD and Implicit ALS) compress interaction matrices into low-rank representations. This compression mathematically favours items with dense interaction histories and pushes niche items to the periphery of the learned space [33, 38].
- **Neighborhood Methods** (User-based k -NN) propagate recommendations through networks of co-occurring items. In sparse catalogs, where many items have very few interactions, the effective neighbourhood of each item shrinks dramatically in the extreme case, the recommender collapses to recommending only a handful of globally popular items to every user, a failure mode this thesis documents empirically [49].
- **Content Models** (TF-IDF profiling and Per-User Logistic Regression) condition predictions on feature similarity rather than interaction volumes. This structural decoupling from

community interaction counts gives content models a theoretical immunity to popularity bias, which this thesis tests under sparse e-commerce conditions [42].

- **Tree Ensembles** (Popularity-Memorising Random Forests) serve as a specialised behavioural baseline. By training a non-linear model directly on item interaction identifiers under extreme sparsity, we can observe how a powerful pattern-recognition system partitions the user population when the only strong signal available in the training data is item popularity [?].

2.1.5 Scope Boundary: Deep Learning and Graph Models

Modern deep learning architectures represent the current state of the art in academic recommender systems research. Neural Collaborative Filtering (NCF) [31] replaces the dot-product similarity of matrix factorization with a learned non-linear neural network, allowing the model to capture more complex user-item interaction patterns. Neural Graph Collaborative Filtering (NGCF) [54] and the simplified LightGCN [32] go further by representing the user-item interaction graph explicitly as a mathematical graph structure and using **graph convolutions** a form of neighbourhood aggregation that propagates information along edges to produce embeddings.

These architectures frequently dominate performance leaderboards on the dense, curated academic benchmark datasets that are standard in the field. Despite this, there are two important reasons why they fall outside the scope of this thesis.

First, a rigorous reproducibility audit found that many neural recommendation approaches do not actually outperform well-tuned classical baselines when evaluated under fair experimental conditions [26]. The apparent superiority often reflects differences in hyperparameter tuning and evaluation protocols rather than genuine architectural improvements. This calls into question whether the complexity of deep architectures is justified in applied settings.

Second, and more critically for this thesis: graph convolutional recommenders have been shown to *amplify* popularity bias substantially more than simpler linear baselines [18]. The mechanism is structural. Graph convolution works by allowing each node (user or item) to aggregate information from its neighbours in the interaction graph. Popular items have vastly more neighbours (they appear in far more users' interaction histories), so they receive information from many more sources during each convolution step. Over multiple convolution layers, popular items accumulate disproportionately rich, well-refined embeddings, while niche items connected to only a few users receive very little signal. Popularity rapidly dominates the entire embedding space.

For the purposes of this thesis, deep learning architectures are deliberately set aside. To accurately diagnose the structural mechanisms through which popularity bias propagates, and to correctly interpret the distributional metrics used to quantify it, we require models whose failure modes are directly traceable to specific, identifiable components of the pipeline. When a simple neighborhood model collapses under severe catalog sparsity, recommending only two or three items to every user, we can explain exactly *why* this happens in terms of the co-occurrence statistics and the shrinking neighborhood structure. This kind of mechanistic interpretability would be far more difficult to achieve with deep architectures. The empirical contributions of this thesis are therefore orthogonal to the goal of chasing marginal accuracy improvements on dense, artificial benchmark datasets we prioritise algorithmic transparency and diagnostic clarity under extreme, realistic stress conditions.

2.1.6 Identified Research Gap

While the individual mechanics of collaborative and content-based scoring are thoroughly documented in the literature when studied in isolation [?], far less work has evaluated them *jointly* under a single unified experimental protocol one that simultaneously measures macro-level item

exposure across the full catalog [27], user segment disparities in recommendation quality [5], and the longitudinal evolution of both through feedback loops [43].

This omission is especially pronounced when the systems under study are subjected to the kind of extreme sparsity characteristic of real-world e-commerce platforms as opposed to the relatively dense benchmark datasets (such as MovieLens) that dominate the academic literature. Furthermore, the existing popularity bias literature has concentrated heavily on proposing and evaluating mitigation techniques and post-processing corrections [2, 12]. There is a clear and underserved need for rigorous empirical studies dedicated strictly to *quantifying* the inherent bias behaviours of baseline algorithms across diverse, ecologically valid dataset domains, before any correction is applied [37]. Section 2.8 returns to this structural gap and defines it in precise terms.

2.2 Theoretical Foundations of Bias

To understand why recommender systems are structurally predisposed to favour popular items, we need to start at the level of the data they are trained on. The core problem is straightforward to state but has deep consequences: the interaction logs that recommender systems learn from are not a neutral, unbiased record of what users actually prefer. They are a heavily distorted sample distorted in predictable, systematic ways that algorithms then learn from and reproduce.

The Missing-Not-At-Random Problem

When a user does not interact with an item does not click on it, does not purchase it, does not rate it there are several very different reasons why that might be true. The most optimistic interpretation is that the user saw the item and was not interested. But in practice, a far more common reason is that the user was simply *never shown the item in the first place*. On most commercial platforms, users are exposed to only a tiny fraction of the full product catalog. The items they interact with are largely determined by what prior recommendation and search systems have surfaced to them. This creates a fundamental problem: the absence of an interaction is not informative about the user’s true preference it may be entirely about *exposure*, not preference.

In statistical terminology, this situation is described as **Missing Not At Random (MNAR)** [41, 48]. Data is said to be **Missing Completely At Random (MCAR)** when the probability that a data point is missing has nothing to do with its value like randomly dropping rows from a spreadsheet. Data is **Missing At Random (MAR)** when missingness depends only on observed variables, not on the missing value itself. MNAR is the hardest case: the probability that a data point is missing depends on the value of the data point itself. In recommenders, whether a user-item interaction is recorded depends partly on whether the user would have liked the item because popular, well-liked items get promoted more and are therefore seen more. The missingness is entangled with the very preference signal we are trying to estimate.

Marlin and Zemel [44] formalised this problem for collaborative filtering, proving that the standard assumption of treating all missing interactions as equally uninformative is incorrect. Missing ratings carry information about both the user (e.g., users who tend to rate items are systematically different from those who do not) and the item (e.g., items that go unrated tend to be niche, obscure, or simply un-surfaced). Ignoring this structure leads to systematically biased estimates of user preferences.

Steck [51] demonstrated a sharper and more practically alarming consequence: optimising a model’s error on MNAR training data does not, in general, minimise error on the unbiased preference distribution one actually cares about. Put plainly a model can appear to perform well on its training objective while simultaneously becoming worse at the true goal of predicting genuine user preferences. This is not a minor statistical footnote; it fundamentally undermines the standard train-and-evaluate pipeline that the majority of recommender system research relies

on.

Why E-Commerce Interaction Logs Are MNAR by Construction

In an e-commerce context, the interaction log $\mathcal{D}$ the complete record of which users purchased, clicked on, or viewed which products is MNAR by construction, not by accident. Here is the feedback loop that creates this:

1. A legacy recommendation or search system surfaces certain items prominently to users.
2. Users interact primarily with those prominently surfaced items.
3. The resulting interaction log is therefore dominated by items that the prior system chose to promote.
4. A new model trained on this log learns, at least in part, the *exposure policy* of the old system the pattern of what got shown rather than the true underlying preferences of users.
5. The new model then surfaces similar items prominently, and the cycle repeats.

This means that algorithms trained on $\mathcal{D}$ risk learning which items were historically promoted as much as they learn what users genuinely prefer. The distinction matters enormously: a historically popular item may have high interaction counts because it was aggressively promoted in the past, not because it is inherently superior to niche alternatives that were never given comparable visibility.

Accuracy Metrics Can Hide This Problem

Steck [52] further showed that standard accuracy metrics such as Root Mean Square Error (RMSE) on held-out ratings can conflate genuine predictive skill with the simpler strategy of conforming to popularity. A model that learns to recommend whatever is most popular will appear to perform well by standard accuracy metrics, because popular items are overrepresented in the test set (for the same MNAR reasons). The metric rewards the model not for understanding user preferences but for reflecting the existing popularity distribution in the data.

This has a practical implication for interpreting benchmark results: a model that ranks highly on RMSE or Precision@K may be doing so primarily because it has learned to recommend popular items, not because it is meaningfully personalised. This is one of the central reasons this thesis focuses on distributional and inequality-based metrics such as Gini coefficients and catalog coverage in addition to standard accuracy scores.

Principled Debiasing: IPS and Why This Thesis Focuses on Measurement Instead

The theoretically principled response to MNAR training data is to model the *propensity* of each interaction that is, to estimate the probability that a given user-item pair would have been observed under a random exposure policy, and then reweight observations accordingly. This approach is called **Inverse Propensity Scoring (IPS)** [50]. By upweighting interactions that were unlikely to have been observed (typically niche items surfaced by chance) and downweighting those that were near-certain to appear (popular items that are always promoted), IPS corrects for the structural bias in the training data and provides a statistically principled route to learning unbiased preference models.

IPS remains the gold standard for debiasing under explicit exposure models. However, this thesis does *not* implement it. The reason is that IPS requires a reliable estimate of exposure propensities, which in turn requires either a known, controlled exposure policy (rarely available

in practice) or strong assumptions about the data-generating process. On the ultra-sparse retail log examined in this study where most users have interacted with only a tiny sliver of the catalog such estimates would be highly unreliable.

More fundamentally, the contribution of this thesis is not to propose a new debiasing algorithm. It is to *measure and compare*: to rigorously quantify how far different algorithmic architectures push recommendations along three dimensions concentration of exposure on a few items, coverage of the full catalog, and disparity in recommendation quality across user segments under realistic MNAR sparsity conditions. The MNAR framework provides the theoretical vocabulary for interpreting what we observe. When, for example, ALS (Alternating Least Squares) for implicit feedback becomes numerically unstable on an ultra-sparse retail log, or when a neighborhood-based model collapses to recommending only two or three items platform-wide, these are not arbitrary software bugs or implementation failures. They are *predictable, theoretically grounded* consequences of applying implicit-feedback objectives and co-occurrence-based similarity to data with pathologically thin support data that is extremely sparse in a non-random way. The MNAR lens makes these failures legible and interpretable.

2.3 Popularity and Exposure Bias: The Propagation Chain

Having established that the training data for recommender systems is structurally biased, this section examines what happens when algorithms are trained on that data: how popularity bias enters the system, how it is amplified through the recommendation pipeline, and how it compounds over time.

Economic Grounding: The Long Tail and Why Bias Works Against It

The commercial promise of digital recommendation was closely tied to the concept of the **long tail** [7] the observation that digital platforms, unlike physical retailers constrained by shelf space, can stock enormous catalogs and profitably serve niche demand. While a physical bookstore stocks a few thousand titles, an online retailer can stock millions. The economic argument was that, in aggregate, the many niche items in the “tail” of the popularity distribution represent a substantial revenue opportunity, even if each individual niche item sells rarely.

Brynjolfsson et al. [13] provided econometric evidence that this long-tail effect is real and commercially meaningful for large online retailers: niche sales, when aggregated across the full tail of the distribution, can rival or exceed revenues from a smaller number of popular items. Recommender systems were supposed to be the engine enabling this: by helping users discover items they would never have found through browsing alone, recommendation could unlock the economic value of the long tail.

Popularity bias directly undermines this promise. When a recommender systematically directs traffic toward already-popular items, it concentrates visibility and revenue on the head of the popularity distribution the blockbuster products and deprives the long tail of the exposure it needs to generate sales. The algorithm, in effect, works against the platform’s economic interest in catalog diversity, as well as against the interests of producers of niche items who depend on recommendation for discoverability.

Early Measurement and the Emergence of Bias-Aware Evaluation

Awareness of popularity concentration as an evaluation problem emerged early in the recommender systems research community. Celma and Herrera [16] introduced novelty- and diversity-oriented evaluation metrics, arguing that a recommender producing only obvious, already-popular recommendations provides low informational value to users even if it achieves high accuracy scores. A system that always recommends the top ten best-selling products might achieve reasonable precision, but it offers nothing that users could not discover on their own.

Abdollahpouri et al. [2] articulated the mechanism behind what they termed the **popularity amplifier**: recommendation models do not simply score items by predicted relevance. They inherit the statistical structure of their training data, in which popular items have been observed far more frequently. This training-frequency signal acts as an implicit prior, elevating head items beyond what their true relevance to individual users would justify. The model does not need to explicitly favour popular items its training objective inadvertently teaches it to do so.

The workshop paper of Abdollahpouri et al. [3] demonstrated empirically that aggregate accuracy metrics standard measures like RMSE or Precision@K that report a single number averaged across all users and items can coexist with grossly unequal item exposure. A model can achieve satisfactory accuracy while effectively ignoring the majority of the product catalog. This becomes visible only when measuring exposure through alternative metrics:

- **Average Recommendation Popularity (ARP)**: the mean popularity (interaction count) of items that appear in recommendation lists. A high ARP indicates the system is predominantly surfacing well-known items.
- **Gini Coefficient**: a measure of inequality in how often each item in the catalog appears in recommendation lists (discussed further below).
- **Catalog Coverage**: the fraction of the full item catalog that appears in at least one recommendation list across all users. A recommender with 5% coverage is effectively ignoring 95% of available products.

These metrics tell a fundamentally different story about system behaviour than accuracy scores alone, and they are central to the evaluation framework of this thesis.

Survey Consolidation and the Under-Representation of E-Commerce Settings

Recent surveys have synthesised the now-extensive literature on popularity bias in recommenders [37], cataloguing dozens of distinct definitions, measurement approaches, and mitigation families. One of the key findings of these surveys is that the academic literature is heavily skewed toward evaluation on a small number of well-known benchmark datasets most prominently the MovieLens dataset [30] which have interaction density profiles that are substantially richer than those encountered in real-world e-commerce. The average MovieLens user has rated dozens of films; the average user on a retail platform has purchased only a handful of products from a catalog of millions. This difference in sparsity is not merely quantitative it qualitatively changes how algorithms behave and fail. Complementary studies in multimedia recommendation domains, which also exhibit heavy-tailed popularity distributions, have employed Gini-based audits to characterise this concentration [39], providing methodological precedent for the macro-level diagnostics used in this thesis.

Distributional Inequality: The Gini Coefficient as a Bias Metric

The **Gini coefficient** is a standard measure of distributional inequality, originally developed in economics to quantify income or wealth inequality [20]. In the context of recommender systems, it is applied to the distribution of item exposure: each item in the catalog is assigned a value equal to the number of times it appears in recommendation lists across all users, and the Gini coefficient measures how unequally these appearances are distributed.

A Gini coefficient of 0 represents perfect equality every item in the catalog receives the same number of recommendations. A coefficient of 1 represents maximum inequality one item receives all recommendations and every other item receives none. In practice, recommender systems produce Gini values somewhere between these extremes, but values close to 1 indicate severe concentration, where the system is effectively recommending the same small set of popular items to everyone.

One technical nuance worth noting: the Gini coefficient has a well-documented finite-sample bias [21] when computed on a small number of items or users, it systematically underestimates true inequality. This means that when measuring Gini on small catalog slices (for example, within a specific product subcategory), the computed values should be interpreted conservatively, as the true inequality may be somewhat higher than measured. This thesis acknowledges this limitation and applies appropriate care when interpreting Gini estimates on fine-grained subsets of the data.

The Gap: Cross-Algorithm Comparison and Segment-Level Disparities

While macro-level bias metrics such as Gini coefficients, catalog coverage, and ARP are now broadly accepted in the recommender systems research community, two important dimensions of the problem remain understudied. First, direct cross-algorithm comparison of bias profiles evaluating multiple architecturally distinct algorithms on exactly the same dataset and reporting both accuracy and bias metrics side by side is less standardised than one might expect. Most papers evaluate one or two algorithms and focus on proposing a mitigation technique, rather than comprehensively benchmarking the baseline bias landscape. Second, the interaction between macro-level exposure inequality and *segment-level disparities among users* the question of whether certain groups of users (for example, those who prefer niche items) receive systematically worse recommendation quality has received less attention than the item-side perspective. It is to this user-centric fairness dimension that the next section turns.

2.4 User-Centric Fairness and Disparate Impact

The previous section established that popularity bias produces unequal item exposure at a macro, catalog-wide level. This section shifts perspective from items to users: even if we accepted a degree of item-level inequality as commercially justified, there is a separate and equally serious problem. Popularity bias does not hurt all users equally. Users whose genuine preferences align with popular, mainstream items receive recommendations that match their tastes reasonably well. Users whose preferences lean toward niche, less-popular items receive systematically worse recommendation quality the system is structurally less capable of serving them. This asymmetry is the core concern of **user-centric fairness** in recommender systems.

What Do We Mean by Fairness in Recommendation?

Fairness is not a single, well-defined concept it is a family of related but distinct concerns that depend on whose interests we are evaluating and what we consider an acceptable distribution of outcomes. Before examining specific findings in the literature, it is worth being precise about the different axes along which fairness can be defined.

Ekstrand et al. [25] situate fairness in recommender systems along multiple **stakeholder axes**: the system serves not just the end user consuming recommendations, but also producers or providers whose items are (or are not) being surfaced, and the platform itself whose interests include engagement, revenue, and regulatory compliance. A decision that is fair from one stakeholder’s perspective may be unfair from another’s. Burke [14] formalises this tension through a **multi-sided fairness** framework, demonstrating mathematically that optimising a single-objective function for one stakeholder (for example, maximising prediction accuracy for consumers) can systematically harm another (for example, producers of niche items whose products are never recommended). These two stakeholder perspectives map cleanly onto different measurement approaches: macro-level exposure metrics such as Gini coefficients and catalog coverage capture **producer-side equity** (are all items receiving fair access to users?), while per-segment accuracy metrics such as RMSE or NDCG (Normalised Discounted Cumulative Gain a ranking quality metric that rewards placing relevant items higher in the list [35]) capture

consumer-side equity (are all users receiving equally useful recommendations?). Both are necessary; neither alone is sufficient.

Mainstreaminess, Niche Users, and Group-Level Gaps

The most direct operationalisation of consumer-side fairness relevant to this thesis comes from Abdollahpouri et al. [5], who introduce a framework specifically designed to evaluate how popularity bias falls unevenly across different user segments. The central construct is **mainstreaminess**: a score assigned to each user that measures how much their historical interaction profile overlaps with the globally popular items on the platform. A high-mainstreaminess user is someone whose preferences happen to align with what the majority of the platform’s users like they enjoy blockbuster films, bestselling books, or top-selling products. A low-mainstreaminess (niche) user is someone whose preferences diverge from the mainstream they enjoy obscure genres, specialist products, or minority-taste items.

Once users are partitioned into segments based on their mainstreaminess score (typically three groups: low, medium, and high), the evaluation asks: does the recommender system perform equally well for all three groups? The answer, consistently found across datasets and algorithms, is no. Mainstream users receive recommendations that closely match their preferences, because the algorithm has abundant training signal for the kinds of items they like. Niche users receive recommendations that are biased toward popular items they have little interest in, because the algorithm lacks sufficient training signal for their preferred items and defaults to popularity as a fallback.

The key metric for quantifying this disparity is **ΔGAP** (Group Average Popularity gap): the difference in average recommendation popularity between user groups. A large ΔGAP indicates that the system is recommending substantially more popular items to niche users than their actual preferences would justify in effect, penalising users for having non-mainstream tastes. This framework directly informs Experiment B of this thesis, which measures ΔGAP across all six baseline algorithms to establish how different architectural assumptions translate into consumer-side fairness outcomes. Related work extends this perspective to multi-stakeholder and longitudinal settings, examining how popularity-related disparities evolve across user segments over time [4].

Disparate Impact Beyond Protected Attributes

In the broader fairness in machine learning literature, discussions of disparate impact often focus on legally protected demographic attributes: race, gender, age, disability status [9, 23]. However, in the recommender systems context, disparate impact need not be and in practice often cannot be defined solely along these axes. Ekstrand et al. [24] document significant outcome disparities in book recommendation systems linked to the gender of book authors in metadata, showing that subgroup analysis based on item-side attributes is a legitimate and informative form of fairness-sensitive evaluation even in the absence of user demographic data. Similarly, [?] show that standard evaluation protocols themselves embed popularity and demographic biases, meaning that what looks like a “fair” evaluation on aggregate metrics may systematically disadvantage certain user groups before any algorithm has even been applied. This broadens the concept of fairness: any systematic pattern in which a particular segment of users or items receives consistently worse outcomes regardless of whether that segment is a legally protected class warrants scrutiny.

For the purposes of this thesis, the relevant grouping variable is user mainstreaminess. This is not a protected attribute in the legal sense, but it is a meaningful one in the commercial and ethical sense: a platform that consistently fails to serve users with non-mainstream preferences is failing a significant portion of its user base and potentially reinforcing a “rich get richer” dynamic in which only popular items are commercially viable.

The Problem with System-Level Averages

A persistent methodological issue in recommender systems evaluation is the reliance on system-level averages: a single RMSE or Precision@K score averaged across all users and items. Leonhardt et al. [40] argue that such averages systematically mask per-user inequality. A system might achieve an acceptable average accuracy while producing excellent recommendations for mainstream users and terrible recommendations for niche users the two effects cancel out in the aggregate statistic, rendering the disparity invisible. This is sometimes called the **statistical fairness masking problem**: aggregate metrics can look healthy precisely because well-served majority groups dilute the poor outcomes of underserved minority groups.

This motivates the per-user evaluation design employed in Experiment B of this thesis: rather than reporting a single aggregated accuracy score, we construct per-user test sets, compute individual-level metrics, and then compare distributions across user segments using statistical tests (specifically the Mann–Whitney U test, a non-parametric test that compares the distributions of two groups without assuming they follow a normal distribution). This allows us to detect and characterise disparity even when aggregate metrics look acceptable. The extension to group recommendation settings where recommendations are generated for a group of users simultaneously introduces additional complexity, as popularity bias can manifest as a “tax” on niche-preferring members of a group [55], but this multi-user setting falls outside the scope of this thesis.

The Accuracy–Fairness Tension

A recurring theme throughout the fairness literature is that accuracy and fairness are often in tension: optimising a model to maximise predictive accuracy can simultaneously reduce recommendation diversity and worsen user-segment disparities [34]. This tension is not incidental it follows directly from the structure of the problem. Accuracy is maximised by exploiting the strongest signals in the training data, and the strongest signals in MNAR interaction logs are, by construction, signals about popular items. A model that learns these signals well will be accurate on average but unfair to niche users.

Dynamic settings compound this trade-off. Zhu et al. [58] demonstrate that popularity bias interacts with temporal drift in user preferences: as the popularity distribution shifts over time, a model that has heavily encoded historical popularity patterns becomes progressively worse at serving users whose preferences evolve toward newly emerging niche items. This temporal dimension of the accuracy–fairness tension is one reason this thesis includes a longitudinal simulation component (Experiment C), rather than treating bias as a static property of a single snapshot in time. The implication for reporting is clear: presenting only RMSE or ranking accuracy as evidence of system quality is insufficient. Distributional metrics must be reported alongside accuracy scores to provide an honest characterization of who the system is serving well and who it is failing.

2.5 Temporal Dynamics and Feedback Loops

The fairness concerns described in the previous section would be serious enough if popularity bias were a static problem a fixed property of a trained model that remains constant over time. In reality, the situation is considerably worse. Recommender systems do not operate in isolation from the world they are trying to model. They operate in a **closed loop**: the system influences what users see, which shapes what users interact with, which becomes the training data for the next version of the system. Each iteration of this loop can push the system further toward a state in which a smaller and smaller number of popular items receive the overwhelming majority of all recommendation-driven traffic.

To understand why this matters, consider a physical analogy. A microphone placed too close to a loudspeaker picks up the speaker’s output and feeds it back into the amplifier, which drives the speaker louder, which is picked up by the microphone again producing the familiar, escalating screech of audio feedback. The microphone is not broken. The amplifier is not broken. The speaker is not broken. The problem is entirely a property of the loop connecting them. Popularity bias in recommender systems works in exactly the same way: individually reasonable algorithmic decisions, when connected in a closed feedback loop, can produce an escalating concentration of attention on a shrinking set of items. This section reviews the formal evidence for this dynamic, the conditions under which it degenerates most severely, the methodological challenges of studying it, and the conceptual distinctions needed to interpret what “diversity” even means in a dynamic setting.

The Filter Bubble and Economic Evidence of Diversity Loss

Public awareness of feedback-driven diversity loss was substantially shaped by Pariser [46], who coined the term **filter bubble** to describe the phenomenon whereby personalisation systems progressively narrow the information diet of individual users, exposing them primarily to content that confirms and reinforces their existing preferences. The concern was initially directed at news and political information: if an algorithm learns that a user engages more with content that aligns with their existing worldview, and then prioritises that content in future exposures, the user is gradually sealed inside a bubble of self-confirming information. The mechanism, however, is domain-agnostic. The same feedback dynamic applies to product recommendation, music streaming, video platforms, and any setting where a personalisation system mediates access to a large catalog of items.

It is important to note that the filter bubble argument, as originally stated by Pariser, is about the *individual* user’s information environment becoming narrower. This is related to, but distinct from, a *societal*-level concern: that when many users are each sealed in their own filter bubbles, the aggregate effect is a fragmentation of shared information and a reduction in common ground across the population. Both effects matter, but they require different metrics to measure, a distinction we return to below.

Importantly, the economic evidence that recommendation systems can reduce *sales diversity* concentrating commercial demand on fewer items predates both the filter bubble metaphor and the deep learning architectures that currently dominate deployed systems. Fleder and Hosanagar [27] demonstrated, using formal economic modelling grounded in consumer choice theory, that recommendation systems can produce what they term a “blockbuster culture” effect. Even when each individual user is receiving personalised recommendations, the *aggregate* effect on the market can be a concentration of demand on a smaller number of already-popular items and a reduction in the commercially viable diversity of the catalog. Popular items receive more recommendations, which drives more purchases, which makes them appear even more popular in the next round of training data, which drives even more recommendations toward them. The rich get richer.

What makes this finding theoretically important is that it does not require the algorithm to be poorly designed or obviously biased. The diversity reduction can emerge from a system that is working exactly as its designers intended accurately predicting what users are likely to enjoy based on the behaviour of similar users simply because the feedback loop between recommendations and consumption creates a self-reinforcing dynamic that amplifies existing popularity differences over time.

Algorithmic Confounding and the Homogenisation of Users

Chaney et al. [17] formalise the feedback mechanism through the concept of **algorithmic confounding**. The term “confounding” is borrowed from causal inference, where a confounding variable is one that influences both the treatment and the outcome, making it difficult to estimate

the true causal effect of the treatment alone. In the recommender systems context, the “treatment” is the recommendation the system’s decision about which items to show a user. The “outcome” is the user’s subsequent interaction behaviour. The confound is the system itself: because the system determines what users are exposed to, and users can only interact with what they are exposed to, the observed interactions are not a clean measure of user preferences. They are a measure of user preferences *filtered through the system’s prior decisions*.

The consequence of this confounding, as Chaney et al. demonstrate formally, is a process of **user homogenisation**. Here is the mechanism, step by step:

1. At time t , the recommender system shows users a set of recommendations. Because popularity bias is already present in the system, these recommendations are skewed toward popular items.
2. Users interact with the items they are shown. Their interaction histories therefore reflect the system’s biases: they contain more interactions with popular items than the users’ true underlying preferences would produce.
3. The next version of the model is trained on these biased interaction histories. It learns that many different users interact with the same popular items, and updates its similarity estimates accordingly: users look more alike than they truly are, because they have been funnelled toward the same popular items.
4. The updated model, perceiving less diversity among users, produces recommendations that are more similar across the user population.
5. These more homogeneous recommendations expose users to an even narrower range of items, further homogenising their interaction histories. Return to step 2.

Critically, this process reduces **welfare** in a way that is invisible to the system. Users end up consuming items they value less than items they would have discovered under a more diverse exposure policy, but the system has no way to detect this, because it never shows users the alternatives. The counterfactual “what would this user have purchased if shown a different set of items?” does not exist in the training data. The system can only see what users did, not what they would have done under different conditions. This is the fundamental epistemological problem created by the closed feedback loop: the system destroys the very evidence it would need to recognise its own failure.

Bias Amplification Across Feedback Cycles

Mansoury et al. [43] provide a formal model of how popularity bias is amplified across successive feedback cycles, and their analysis yields a result that may be surprising: significant bias amplification can occur even when the algorithm at each individual step is not obviously miscalibrated. The amplification is a property of the *loop*, not of any single model. To understand this intuitively, consider a small numerical example. Suppose that at baseline, the top 10% of items receive 60% of all recommendations a meaningful concentration, but not catastrophic. After one round of retraining on the interactions generated by these recommendations, the top 10% of items now receive 70% of all recommendations, because the training data was itself skewed toward them. After another round, the figure rises to 78%. After a third round, to 84%. No single retraining step involves an obviously wrong decision. The cumulative effect, however, is a system that has effectively abandoned the tail of its catalog.

This finding has a direct methodological implication for this thesis: to characterise how bias evolves under realistic deployment conditions, it is not sufficient to train a model once and

evaluate it on a held-out test set. We must simulate multiple rounds of the recommend-observe-retrain cycle and track how distributional metrics Gini coefficients, catalog coverage, ΔGAP change across iterations. This is the experimental design of Experiment C.

Jiang et al. [36] extend the analysis to identify the conditions under which the feedback loop leads to a fully **degenerate** outcome: a state in which the system’s ranking collapses to a trivial ordering, recommending the same small set of items to every user regardless of their individual preferences. This degenerate state represents a simultaneous failure of personalisation (every user receives the same recommendations), catalog coverage (the vast majority of items never appear in any recommendation list), and user welfare (niche users receive recommendations entirely disconnected from their actual preferences). What makes the degenerate outcome particularly alarming is its path dependence: a system can arrive at this state through a sequence of individually defensible decisions, with no single step being obviously wrong in isolation. The failure is emergent it is a property of the trajectory, not of any snapshot along it.

The specific feedback regime most likely to produce degeneracy, according to Jiang et al., is one in which the recommendation list has a high **position bias**: users are much more likely to interact with items near the top of the list than with items lower down, regardless of the items’ true relevance. Under strong position bias, the items that happen to appear at the top of the list at time t accumulate disproportionately more interactions, making them appear even more popular in the time $t + 1$ training data, which places them even higher in future lists. The feedback between position bias and popularity bias is mutually reinforcing, and can rapidly drive the system toward the degenerate state.

The Matthew Effect: A Sociological Lens

The dynamics described above in which items that are already popular accumulate further popularity advantages through feedback, while less popular items are progressively starved of exposure are a specific instance of a general phenomenon in social systems known as the **Matthew Effect**, named after the Biblical verse “to him who has, more shall be given” and formalised sociologically by Merton [45]. Merton documented this cumulative advantage dynamic in the context of scientific citation: well-cited papers accumulate further citations disproportionately, not only because they are genuinely more valuable, but because they are more visible, and visibility itself drives further citation. The parallel to recommendation is precise: popular items are more visible (they are recommended more frequently), their increased visibility drives more interactions, those interactions increase their measured popularity, and their increased popularity drives further recommendations. The feedback between popularity and visibility is the engine of the Matthew Effect in recommender systems, and it operates entirely independently of whether popular items are genuinely superior to niche alternatives.

Recognising the Matthew Effect as the underlying social dynamic helps clarify why popularity bias is not merely a technical calibration problem to be fixed by a better loss function. It is a structural property of systems that use observed popularity as a signal for future promotion. Any system that does this regardless of its specific algorithmic details will be subject to Matthew Effect dynamics unless it explicitly corrects for them.

Methodological Considerations for Simulation-Based Evaluation

Given the ethical and practical constraints on conducting longitudinal randomised experiments with real users, simulation-based evaluation is the primary empirical tool for studying feedback dynamics in recommendation research. The general approach involves: (1) training a recommendation model on a real historical interaction dataset, (2) using the model to generate recommendations for a simulated user population, (3) generating synthetic interactions according to a click model that specifies how users respond to recommendations, (4) retraining the

model on the synthetic interactions, and (5) repeating steps 2–4 for multiple cycles, measuring distributional metrics at each step.

The validity of conclusions drawn from this procedure depends critically on the realism of the click model used in step 3. Yao and Huang [56] provide a careful analysis of the assumptions that must hold for simulation-based evaluation to produce valid inferences about real-world system behaviour. They identify three key modelling choices that have the largest influence on results:

1. **Position bias model:** How much does the rank of an item in the recommendation list affect the probability of a click, independently of the item’s true relevance? Empirically, position bias is strong: items ranked first receive far more clicks than equally relevant items ranked fifth, purely due to their position. The choice of position bias model significantly affects how quickly feedback loops amplify popularity bias.
2. **Exploration rate:** Do simulated users ever interact with items outside the recommendation list searching independently, following external links, or browsing the catalog directly? The degree to which users explore beyond the algorithm’s suggestions determines how “sealed” the feedback loop is. A system where users never explore outside recommendations creates a maximally closed loop; a system with substantial external discovery leaves the loop partially open, slowing the amplification of popularity bias.
3. **Preference drift:** Do user preferences change over time in response to repeated exposure? If users gradually develop a preference for items they are repeatedly exposed to a form of habituation or taste shaping then the feedback loop affects not just what users consume but what they *want*, making the welfare loss from homogenisation even harder to measure and reverse.

Experiment C of this thesis explicitly documents and justifies each of these modelling choices, following the methodological transparency guidelines proposed by Yao and Huang [56], so that readers can assess how sensitive the conclusions are to the specific simulation assumptions made.

Individual Versus Aggregate Diversity: A Critical Distinction

A conceptual distinction that is essential for correctly interpreting the results of feedback-loop simulations is the difference between **individual diversity** and **aggregate diversity**, documented by Fleder et al. [28]. These two quantities are related but genuinely distinct, and a recommendation system can improve one while simultaneously degrading the other.

Individual diversity refers to the breadth of a single user’s recommendation list or consumption portfolio: does this particular user receive (and consume) a diverse range of items, or is their experience dominated by items from a single genre, category, or popularity tier?

Aggregate diversity refers to the breadth of the full recommendation system’s output across *all* users: across all the recommendation lists generated for the entire user population, how many distinct items from the catalog appear? A system with high aggregate diversity ensures that many different items receive exposure, even if no single user’s list is particularly broad.

To see why these two quantities can diverge, consider the following scenario. A recommendation system notices that every user in its population has interacted primarily with items from their own niche. To improve individual diversity, the system begins recommending a small set of “crossover” items that are popular within multiple different niches to users across the platform. Each individual user now receives a more diverse list, because it includes both their core niche items and some crossover items. But those crossover items are the same for almost every user. The aggregate diversity of the system’s output actually *decreases*: across all recommendation lists, fewer distinct items from the full catalog appear, because the crossover items are crowding out the niche items that were previously unique to different user segments.

Individual diversity went up. Aggregate diversity went down. Standard metrics that report only one of these quantities would give a misleading picture.

This distinction is particularly consequential when comparing the feedback-loop trajectories of collaborative and content-based algorithms, because the two paradigms differ structurally in the mechanism by which they select “bridge” items to expand a user’s recommendation list. Collaborative methods bridge users through item co-occurrence networks, which tend to funnel attention toward the same globally popular crossover items. Content-based methods bridge through feature similarity, which can in principle route different users toward different niche items tailored to their specific taste profiles. Whether this structural difference translates into meaningfully different aggregate diversity trajectories under the closed feedback loop is an empirical question that Experiment C is designed to answer.

Recent large-scale observational evidence from a major music streaming platform [8] provides real-world context for this nuance. The study finds that the relationship between recommendation and listening diversity is more complex than either the optimistic long-tail narrative (recommenders help users discover niche content) or the pessimistic filter-bubble narrative (recommenders seal users into narrowing taste bubbles) would predict. The direction and magnitude of the effect depend on user characteristics, catalog structure, and the specific algorithmic approach reinforcing the case for the kind of disaggregated, algorithm-specific analysis this thesis undertakes.

The Gap This Thesis Addresses

The literature reviewed in this section establishes, with reasonable empirical and theoretical confidence, that popularity bias compounds through feedback loops in collaborative filtering systems. What remains comparatively underexplored is how this dynamic plays out *differently* across algorithm families with structurally distinct inductive biases specifically, whether the theoretical immunity to popularity concentration that content-based methods possess by virtue of their decoupling from interaction-frequency signals survives repeated retraining on recommendation-generated data.

Most existing feedback-loop studies examine a single algorithm or a single class of algorithms in isolation, making it impossible to determine whether the observed dynamics are a property of the specific algorithm, the simulation assumptions, or the broader algorithmic paradigm. A controlled, cross-algorithm comparison subjecting all candidate algorithms to identical click-generation conditions, the same user population, the same catalog, and the same retraining schedule would allow direct causal attribution of the observed feedback trajectories to differences in algorithmic structure. This is the contribution of Experiment C: by holding all simulation parameters constant and varying only the recommendation algorithm, we can isolate the effect of architectural inductive biases on long-run bias amplification, and determine whether content-based methods offer meaningful protection against feedback-driven catalog concentration under the extreme sparsity conditions characteristic of the Takealot dataset.

2.6 Mitigation strategies: a methodological survey

Once popularity bias has been identified and measured, the natural next question is: what can be done about it? Researchers have proposed a wide variety of techniques to reduce or counteract popularity bias in recommender systems. To make sense of this landscape, it is helpful to organise these techniques according to *when* in the recommendation pipeline they intervene. There are three broad stages at which a mitigation can act: **before** the model is trained (pre-processing), **during** the model’s training (in-processing), or **after** the model has already produced a ranked list of recommendations (post-processing). Each stage has its own strengths, limitations, and practical trade-offs. This section surveys representative techniques at each stage and explains

the intuition behind them in plain terms.

It is worth clarifying upfront why mitigation is necessary at all, rather than simply collecting better data. As established in Section 2.2, the interaction logs that recommender systems learn from are not a neutral sample of user preferences they reflect, in part, which items the platform has historically chosen to promote. This means the bias is *structural*: it is baked into the data before any algorithm has even seen it. No matter how carefully an algorithm is designed, if the training data systematically over-represents popular items, the model will inherit that skew to some degree. Mitigation strategies attempt to correct for this inherited skew, either by cleaning up the data before training, adjusting how the model learns from the data, or reshaping the model’s output before it is shown to users.

Pre-processing: correcting the data before training

Pre-processing mitigations intervene *before* a model is trained, by modifying the training data itself so that the model receives a less skewed picture of user preferences. The core intuition is straightforward: if the training data over-represents popular items, we can attempt to rebalance it either by reducing the influence of interactions involving very popular items, or by upweighting interactions involving niche items so that the model does not learn an overly popularity-distorted representation of user tastes.

One family of pre-processing techniques involves **reweighting** individual interactions. Rather than treating every user-item interaction as equally informative, the training procedure assigns a numerical weight to each observation. Interactions involving items that have appeared very frequently in the training data popular items receive lower weights, so the model “pays less attention” to them during training. Interactions involving rare or niche items receive higher weights, so the model devotes more of its learning capacity to getting those right. The challenge is deciding *how much* to upweight or downweight each interaction, which requires assumptions about the true underlying exposure distribution assumptions that are difficult to verify in practice [12].

A related approach involves **rebalancing** the training set directly for example, by subsampling interactions involving popular items, or by artificially augmenting interactions involving niche items through data synthesis. Zhu et al. [59] introduce the concept of “popularity-opportunity bias,” pointing out that popular items not only appear more frequently in training data but also have more *opportunities* to be interacted with simply because they are shown to more users. Their mitigation corrects for this by adjusting the sampling distribution of training pairs to reflect the underlying opportunity structure more fairly. Think of this as levelling the playing field: if popular items have had ten times more chances to collect interactions than niche items, simply comparing raw interaction counts is misleading. The correction tries to evaluate all items as if they had been given comparable opportunities to be discovered.

The appeal of pre-processing mitigations is that they are model-agnostic: once the rebalanced training set is prepared, any recommendation algorithm can be trained on it without modification. This makes them easy to plug into existing pipelines. Their key limitation is that they cannot recover information that was never collected in the first place. If a niche item was never shown to users and therefore has zero interactions in the training data no amount of reweighting can create a meaningful signal for the model to learn from. Pre-processing can attenuate the *distortion* of the data; it cannot conjure *absent* data.

In-processing: modifying how the model learns

In-processing mitigations go one level deeper: rather than modifying the data fed to the model, they modify the model’s **objective function** the mathematical criterion the model optimises during training. The intuition is that, since standard training objectives (such as minimising squared prediction error) reward the model for getting popular items right because there are so

many of them we can redesign the objective to explicitly penalise or correct for this popularity-driven incentive.

One of the most theoretically principled in-processing techniques is **Inverse Propensity Scoring (IPS)** [50]. To understand IPS, it helps to think about what “propensity” means in this context. The propensity of a user-item interaction is the probability that this interaction *would have been observed* under a hypothetical unbiased exposure policy one where all items had equal chances of being shown to users. In reality, popular items have high propensity (they are almost certain to appear in training logs because they are shown everywhere), while niche items have low propensity (they rarely appear in logs because they are rarely surfaced). IPS corrects for this imbalance by reweighting each training observation inversely proportional to its propensity: interactions that were *very likely to be observed* (popular items) are downweighted, while interactions that were *unlikely to be observed* (niche items) are upweighted. This produces a loss function that, in expectation, treats all items as if they had been exposed equally. Applied to recommender systems, IPS can substantially reduce the popularity bias inherited from MNAR training data [50].

The practical challenge with IPS is estimating the propensity scores themselves. In a controlled experiment where a platform deliberately randomises which items are shown to users propensities can be computed exactly. In real deployment, where the exposure policy is complex, historical, and only partially observed, propensity estimation requires additional assumptions and modelling effort. On ultra-sparse datasets like the Takealot corpus used in this thesis, where the majority of items have very few or zero interactions, propensity estimates become highly unreliable. This is one reason IPS is not implemented in the present study, as discussed in Section 2.2.

A second family of in-processing techniques modifies the training objective through **popularity-aware regularisation**. In machine learning, regularisation refers to any technique that penalises the model for becoming “too complex” or for placing too much emphasis on certain patterns in the training data it is a way of preventing the model from overfitting. Popularity-aware regularisers add an explicit penalty to the training objective that discourages the model from assigning disproportionately high scores to popular items, even when doing so would reduce training error. Abdollahpouri et al. [1] apply this idea to learning-to-rank matrix factorisation: their regulariser penalises configurations of the latent factors that would systematically rank popular items above less popular items beyond what user preferences alone would justify. This forces the model to “work harder” to justify recommending popular items, making it more willing to surface niche alternatives when they are genuinely a good fit for the user.

An important nuance that in-processing research has uncovered is that not all popularity signal is harmful or worth suppressing. Zhao et al. [57] make a careful distinction between what they call **benign** and **harmful** popularity bias. Benign popularity bias refers to the case where an item is popular *because* it is genuinely good it has high intrinsic quality, broad appeal, and satisfies the preferences of a large number of users. In this case, the model’s tendency to recommend popular items is not a flaw; it is a reasonable reflection of actual user preferences. Harmful popularity bias, by contrast, refers to the case where an item receives recommendations primarily because it was historically over-exposed or aggressively promoted, not because it is intrinsically superior to alternatives. Suppressing this kind of popularity is desirable. The challenge and Zhao et al. [57]’s key contribution is developing techniques to disentangle these two components from the training data, so that the model can retain the useful popularity signal while discarding the artifactual one. This distinction is especially relevant in this thesis when interpreting the feedback simulation results of Experiment C, where the stochastic click model intentionally retains a partial popularity channel (weighted 0.3 in the blend) to reflect the realistic tendency of users to click on items they have encountered before, even if those items are not perfectly matched to their tastes.

In-processing mitigations are generally more powerful than pre-processing ones because they can reshape the model’s learned representations the latent factors or embeddings it uses to

characterise users and items rather than merely adjusting the data the model sees. However, they are also more tightly coupled to a specific model architecture: a regulariser designed for matrix factorisation cannot straightforwardly be applied to a content-based pipeline or a neural architecture without redesign.

Post-processing: reshaping the output after the model has decided

Post-processing mitigations operate at the final stage of the pipeline: they take a ranked list of recommendations that a trained model has already produced and **reorder or filter** that list to correct for popularity bias before it is presented to the user. Crucially, post-processing does not change the underlying model at all: the model’s weights, embeddings, and internal representations remain fixed. The intervention acts only on the *output*, transforming the initial ranked list into a more balanced one.

The most widely studied post-processing approach is **personalised re-ranking**. The basic procedure is as follows: first, the recommendation model generates a large candidate set of items it considers potentially relevant for the user (say, the top 50 items by predicted score). Then, a second-stage algorithm reorders this candidate set by applying a modified scoring function that penalises items according to their popularity. Items that are very popular receive a penalty that reduces their effective score, giving less popular items a better chance of rising to the top of the final recommendation list. The strength of the penalty is typically controlled by a tunable parameter (often called λ), which allows system designers to control the trade-off between recommendation relevance and diversity. Abdollahpouri et al. [2] develop a personalised variant of this approach in which the penalty is calibrated individually for each user based on how much their current recommendation list already resembles the global popularity distribution. This personalisation is important: a user who already prefers mainstream items should not be forced to receive niche recommendations they do not want, while a user with niche tastes should receive a stronger correction that better reflects their preferences. This technique is a direct methodological antecedent to the SVD+Rerank baseline evaluated in Experiment A of this thesis (Section 4), which applies a popularity penalty with $\lambda = 0.8$ on top of vanilla SVD scores.

A second post-processing approach is **calibration** [53]. Calibration addresses a specific failure mode of recommendation ranking: the final ranked list may not reflect the user’s true preference distribution. For example, if a user’s interaction history contains 60% action films and 40% documentaries, an idealised recommendation list should maintain roughly the same proportions. In practice, recommendation models tend to collapse this diversity: they may recommend almost exclusively the category with the highest predicted scores, even if the user clearly enjoys both. Calibration mitigations explicitly enforce that the distribution of item attributes (genre, popularity tier, niche level) in the recommendation list matches the distribution in the user’s interaction history. This ensures the list is a faithful representation of the user’s breadth of taste rather than a collapsed, popularity-driven approximation of it.

A third family of post-processing techniques uses **pairwise fairness constraints** applied at ranking time [11]. Rather than penalising individual items for their popularity, these methods enforce constraints on *pairs* of items in the ranked list: for example, requiring that, in expectation, a niche item should not be ranked dramatically lower than a popular item of similar predicted relevance. This framing connects recommender system fairness directly to the broader fairness-in-machine-learning literature [9, 23], where fairness is often defined in terms of constraints on relative outcomes between groups rather than on absolute outcome values. Further advances in this space include methods that promote fair exposure across item groups while maintaining overall ranking quality [24, 25].

Post-processing mitigations have several practical advantages that make them attractive in industrial settings. Because they operate on top of an already-trained model, they can be applied to *legacy systems* that were not designed with fairness in mind, without requiring retraining or access to the model’s internal parameters. They are also interpretable: a designer can inspect the

before-and-after ranked lists and understand exactly what changed and why. For these reasons, post-processing is often the first intervention considered when a deployed recommender system is found to exhibit unacceptable bias levels.

However, post-processing carries a fundamental limitation that is central to the argument of this thesis. Because the underlying model’s parameters are unchanged, the model’s *internal representation* of users and items remains shaped by the biased training data. The post-processing step can correct for popularity in the final output list, but it cannot fix the fact that the model’s latent space has been distorted by MNAR training data. Niche items still have poorly estimated embeddings; niche users still have sparse, unreliable preference profiles. A post-processing step can push a niche item higher in the ranked list, but if that item’s embedding was poorly estimated to begin with because the model had very little training data for it the underlying relevance signal being amplified is itself unreliable. This is precisely why Section 2.2 emphasises that MNAR analysis and measurement remain essential even when post-processing interventions are applied: understanding the source and severity of bias in the training data is a prerequisite for correctly interpreting what any downstream mitigation can and cannot achieve.

Positioning this thesis within the mitigation landscape

This thesis does not propose a new mitigation algorithm. Its primary contribution is *measurement and diagnosis*: establishing a rigorous empirical baseline that characterises how severely different algorithmic architectures exhibit popularity bias across three measurement scales (macro exposure, user-segment fairness, and longitudinal feedback dynamics), and across dataset domains ranging from dense academic benchmarks to an ultra-sparse retail environment. This kind of systematic baseline study is a necessary precondition for effective mitigation design you cannot correct a bias you have not yet measured, and you cannot know which mitigation is appropriate until you understand which algorithmic and data-structural mechanisms are driving the bias in your specific setting [37].

The exception is the lightweight SVD+Rerank post-processing variant evaluated in Experiment A, which serves not as a proposed solution but as an *existence proof*: evidence that even a simple, parameter-light post-processing step can meaningfully shift the macro-level exposure distribution without sacrificing rating accuracy. This constrains the lower bound of what practitioners can achieve with minimal engineering effort, and motivates the stronger corrections that more principled in-processing approaches can offer.

2.7 Theoretical framework

The preceding sections of this chapter have reviewed a wide range of topics: how collaborative and content-based recommender systems work, why the data they learn from is structurally biased (MNAR), how popularity bias manifests at the level of the full item catalog, how it falls unevenly across different user segments, how it compounds over time through feedback loops, and what kinds of mitigation strategies researchers have proposed. Taken individually, each of these threads tells an important but incomplete story. Taken together, they describe a single, coherent process through which bias enters a recommender system, propagates through its architecture, and compounds over time.

This section makes that single process explicit. Rather than treating macro-level exposure concentration, segment-level disparities in recommendation quality, and longitudinal diversity loss as three separate “bias problems,” this thesis frames them as three different *observational scales* on one underlying mechanism: the **Bias Propagation Chain**. The purpose of naming and formalising this chain is not merely conceptual tidiness. It is to provide a principled theoretical backbone that links the three research questions (RQ1–RQ3) and three experimental chapters (A–C) into a coherent empirical programme rather than a collection of loosely related studies.

The chain draws on ideas from several distinct intellectual traditions. From **sociology**, the concept of cumulative advantage sometimes called the “Matthew Effect” after the biblical observation that “the rich get richer” describes how small initial differences in visibility or resources can compound into large, self-reinforcing inequalities over time [45]. From **missing-data theory**, the MNAR framework formalises why the absence of an observation is itself informative and cannot be treated as random noise [41, 48]. From **causal exposure modelling**, inverse propensity scoring provides a language for describing how observed interactions are shaped by the exposure policy of the platform, not just by user preferences [50]. From **fairness-aware machine learning**, the literature on disparate impact and individual fairness provides the normative vocabulary for evaluating whether a system’s outcomes are equitable across groups [9, 23]. The Bias Propagation Chain synthesises these traditions into a single model that is concrete enough to be operationalised in experiments while remaining general enough to apply across different algorithmic architectures and dataset domains.

2.7.1 The Bias Propagation Chain model

The central claim of this theoretical framework is that popularity bias in recommender systems is not a static property of a single model or a single dataset. It is a *dynamic process* that unfolds across several coupled stages, each of which feeds into the next. To describe this process precisely, we need a small amount of formal notation, which is introduced here and used consistently throughout the remainder of the thesis.

Let $\mathcal{D}$ denote the **logged user–item interaction matrix**: the complete record of which users interacted with which items, as observed and stored by the platform. Each entry in $\mathcal{D}$ represents a recorded interaction a click, a purchase, a rating, or a play count between a specific user and a specific item. The key word here is “recorded”: $\mathcal{D}$ does not contain the full truth about user preferences. It contains only those interactions that the platform happened to observe, which, as established in Section 2.2, is a heavily non-random subset of all the interactions that *would have occurred* if every user had been exposed to every item equally.

This is the starting point of the chain. Following the missing-data framework of Rubin [48] and Little and Rubin [41], the missingness in $\mathcal{D}$ is not innocuous. Under MNAR conditions, the probability that a given user-item pair appears in $\mathcal{D}$ depends on both the user’s latent preference for that item *and* on whether the platform’s prior exposure policy happened to surface that item to that user. Popular items which have been promoted widely and consistently have high probability of appearing in $\mathcal{D}$ regardless of whether individual users truly prefer them. Niche items which have rarely been surfaced have low probability of appearing in $\mathcal{D}$ even for users who would genuinely prefer them if they encountered them. The result is a training dataset that is already skewed toward popular items before any algorithm has seen it.

Schnabel et al. [50] formalise the implication of this structure for model training and evaluation: the observed ratings or clicks in $\mathcal{D}$ are best understood as draws from the *intersection* of two distributions the user’s true preference distribution *and* the platform’s exposure policy distribution rather than from the user’s true preferences alone. This intersection is what makes the MNAR problem so difficult: we cannot simply separate “what the user likes” from “what the system showed them” by inspecting $\mathcal{D}$ alone, because $\mathcal{D}$ is itself a product of both.

When a collaborative filtering algorithm whether a neighborhood-based method or a latent-factor model is trained on $\mathcal{D}$, it inherits this skewed support set. The algorithm does not know that popular items appear frequently in $\mathcal{D}$ partly because of the platform’s exposure policy; it simply observes that they appear frequently, and treats this as evidence that they are widely preferred. As a result, the algorithm learns representations of popular items that are richer, more refined, and more confident than those of niche items, which appear rarely and therefore provide the model with little training signal. The algorithm’s **inductive bias** its built-in structural assumptions about how users and items relate then further amplifies this inherited skew. Latent-factor models, for example, reward co-occurrence structure in the data, and co-occurrence is

itself shaped by popularity (popular items appear alongside everything simply by virtue of being ubiquitous). Neighbourhood methods propagate similarity through co-occurrence counts, which are dominated by head items for exactly the same reason [2, 52].

The result, at the **macro scale**, is that the model’s recommendation output is more concentrated on popular items than user preferences alone would justify. This is the algorithmic analogue of cumulative advantage [45]: items that received more historical exposure generate denser training signals, which produce stronger model representations, which generate more future recommendations, which generate yet more interactions. Small initial differences in item visibility compound, through repeated cycles of training and recommendation, into large and self-reinforcing inequalities in item exposure.

The same mechanism, read at the **user scale**, produces a different but related problem. Users whose preferences happen to align with the head of the popularity distribution mainstream users benefit from the dense training signal that popular items provide. The model can characterise their preferences accurately because the items they like appear frequently in $\mathcal{D}$, and the items similar to those also appear frequently, giving the model a rich neighbourhood of evidence to draw on. Niche users, by contrast, prefer items that appear rarely in $\mathcal{D}$. Their collaborative neighbourhoods are thin and unreliable. The model’s predictions for their preferences are systematically noisier and less accurate. These users experience what this thesis terms the **niche tax**: a systematic degradation in recommendation quality that is not a consequence of their preferences being objectively harder to satisfy, but a consequence of the structural bias in the training data and the algorithm’s inductive assumptions [5, 40].

Read **longitudinally**, the chain closes into a loop. When a recommender system is deployed, its recommendations shape which items users interact with next. Those interactions are logged and become the training data for the next version of the model. But this training data is no longer an independent sample of user preferences it is, in part, a product of the previous model’s own recommendation decisions. If the model over-recommended popular items, users interacted with popular items, those interactions were logged, and the next model is trained on a dataset that is *more* popularity-skewed than the original. Over successive retraining cycles, this feedback loop can drive the system toward a state of severe concentration, where a handful of items dominate all recommendation lists for all users [17, 43]. This is the longitudinal manifestation of the chain, and it is what Experiment C of this thesis is designed to study.

These three scales macro amplification, user-segment impact, and longitudinal dynamics are not three separate bias problems. They are three observable consequences of the same underlying process: a recommender system trained on MNAR data, whose inductive biases amplify the skew in that data, whose recommendations shape the next round of interactions, and whose retraining on those interactions compounds the skew further. Figure 2.1 summarises this chain and the feedback loop that closes it.

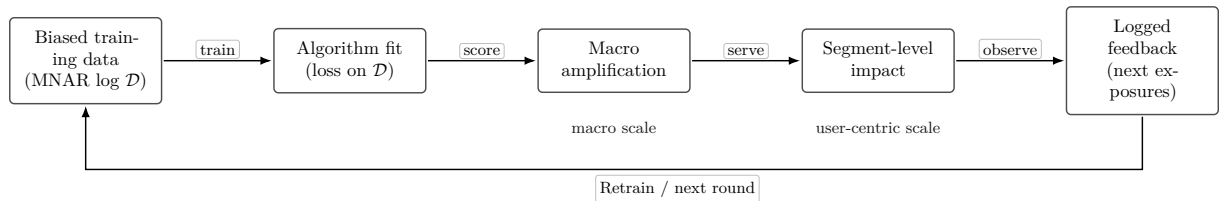


Figure 2.1: The *Bias Propagation Chain*: MNAR training data and exposure-coupled objectives produce macro-level concentration, uneven segment-level service, and feedback-generated logs that re-enter training. Macro diagnostics, user-centric tests, and longitudinal simulation (Experiments A–C) interrogate successive links of the same loop.

2.7.2 Constructs and formal definitions

Before proceeding to the hypotheses and experimental design, it is important to establish precise, consistent definitions for the key constructs used throughout this thesis. The popularity bias literature is unfortunately inconsistent in its terminology: different papers use the terms “popularity bias,” “exposure bias,” and “diversity” in overlapping and sometimes contradictory ways. This creates a risk of conceptual confusion when comparing results across studies. The definitions below fix the meaning of each construct as it is used in this thesis, distinguishing carefully between bias that originates in the *training data*, bias that arises from *how the model surfaces items at decision time*, disparities in *service quality across user segments*, and changes in *aggregate recommendation diversity over time*. These are related but distinct phenomena, and conflating them obscures the causal structure that the Bias Propagation Chain is designed to make explicit.

Popularity bias (training distribution). Let f_i denote the empirical frequency or, in the case of implicit feedback, the raw interaction count of item i in the logged interaction matrix $\mathcal{D}$. Popularity bias is the structural *over-representation* of items with high f_i relative to the reference distribution that would arise under a hypothetical unbiased data-collection process for example, if every user had been randomly exposed to every item with equal probability. This is a property of the *data generating process* that produces $\mathcal{D}$. It exists before any particular recommendation model has been trained, and it is the primary source of skew that all downstream algorithmic decisions inherit. Crucially, items with high f_i are not necessarily *better* than items with low f_i ; they are simply items that the platform’s historical exposure policy happened to surface to more users, giving them more opportunities to accumulate interactions.

Exposure bias (surfacing and missingness). For a given user u , let P_u denote the **latent full-information preference distribution** the distribution over items that describes what this user would prefer *if they had been exposed to every item and had the opportunity to form a genuine opinion about it*. In practice, this distribution is never directly observable. What we observe instead is shaped by E_u , the **platform exposure distribution**: the distribution over items that the platform actually surfaces to user u during the data collection period. The interactions we record for user u are draws from the intersection of P_u and E_u items the user both prefers and was exposed to. Exposure bias names the systematic wedge between P_u (what the user truly prefers) and E_u (what the system surfaces), including the consequence that large regions of the item catalog are simply never represented in $\mathcal{D}$ for most users [44, 50]. This is the recommender-system specialisation of MNAR: the probability that a user-item pair is observed in $\mathcal{D}$ depends on the user’s latent preference for the item *and* on the platform’s historical decision to expose that item to that user. Exposure bias is therefore not reducible to “the model likes popular items.” It is the causal structure upstream of any model, embedded in the data collection process itself, that makes popular items far likelier to appear in $\mathcal{D}$ regardless of true user preferences.

Niche tax (segment-level service gap). Users can be partitioned into cohorts based on how closely their historical interaction profiles overlap with the globally popular items on the platform. This measure is called **mainstreaminess** [5]: a user with high mainstreaminess prefers items that are widely popular across the platform; a user with low mainstreaminess (a “niche user”) prefers items that most other users rarely interact with. The **niche tax** is the systematic difference in recommendation quality measured by metrics such as prediction error (RMSE) or ranking quality (NDCG@ K) between niche users and mainstream users, when both groups are evaluated under the *same* model and experimental protocol. A positive niche tax means niche users receive systematically worse recommendations: their predictions are less accurate, and the items ranked highest for them are less relevant to their actual preferences. The niche tax is an *outcome disparity* metric. It does not measure a property of the data alone or a property of the model alone; it measures the *interaction* between the structural biases in the data and the inductive assumptions of the model, as experienced by different groups of users.

Diversity decay (longitudinal). Let $R^{(t)}$ denote the **multiset** of recommended items

at feedback round t that is, the collection of all items appearing in all recommendation lists generated at round t , aggregated across all users (where the same item can appear multiple times if it is recommended to multiple users). Diversity decay is the reduction, over increasing rounds t , in *dispersion indices* computed on $R^{(t)}$. Dispersion indices are metrics that measure how spread out or concentrated a distribution is. In this context, relevant dispersion indices include catalogue coverage (the fraction of all available items that appear in at least one recommendation list) and the Gini coefficient of item recommendation frequencies. A system exhibiting diversity decay shows falling catalogue coverage and rising Gini over successive feedback rounds, holding the simulation protocol constant [17, 43]. Diversity decay is a *dynamic* construct: it can only be measured over time and requires a longitudinal evaluation protocol, which is why Experiment C of this thesis employs a multi-round feedback simulation rather than a single-snapshot evaluation.

Catalog starvation (extreme macro manifestation). Catalog starvation is the *limiting case* of diversity decay: the regime in which the recommended support the set of items that ever appear in any recommendation list collapses to a vanishing fraction of the full catalog. In practical terms, a system in a state of catalog starvation is recommending the same tiny set of extremely popular items to virtually every user, regardless of their individual preferences. This represents a complete failure of the system’s core function: personalised discovery. What makes catalog starvation particularly insidious is that standard accuracy metrics can remain deceptively stable even as it occurs. If the few items being recommended happen to be genuinely popular with most users, average RMSE and Precision@K can look acceptable on aggregate the aggregate metric is dominated by the majority of users whose mainstream tastes are being served while niche users receive uniformly poor recommendations and the vast majority of the catalog is effectively invisible [52]. Catalog starvation is the macro-level endpoint of the Bias Propagation Chain under conditions of extreme sparsity and strong collaborative amplification, and it is documented empirically for the Takealot corpus in Experiment A (Chapter 4).

Figure 2.2 situates these five constructs in relation to one another, showing how training-side skew and decision-time missingness feed segment-level disparities, which feed longitudinal dynamics, with catalog starvation as the extreme limiting outcome.

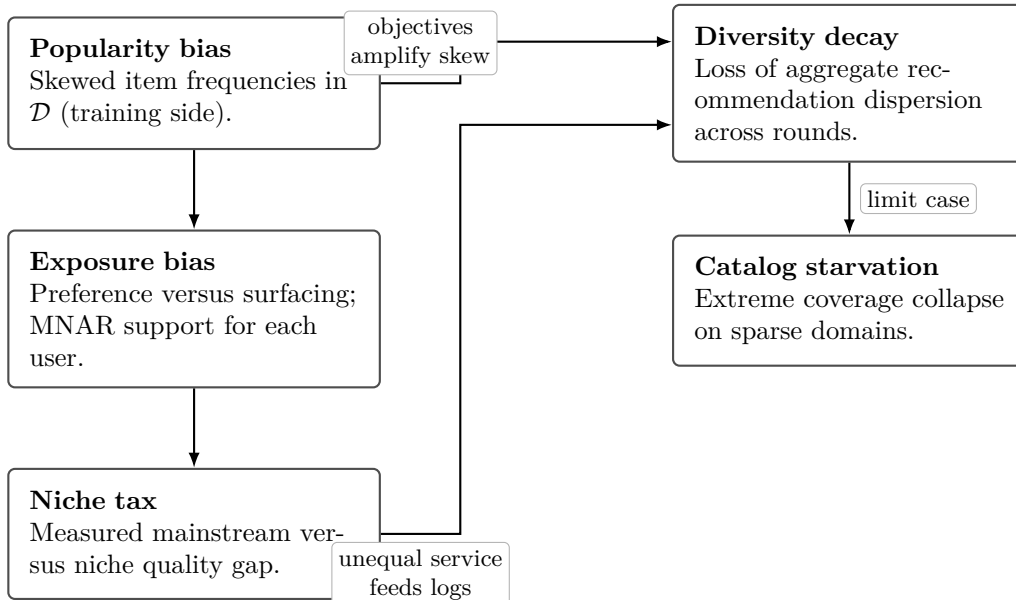


Figure 2.2: Relations among constructs used in this thesis: training-side popularity skew and exposure-coupled missingness shape segment-level disparities; feedback and retraining link user-level outcomes to longitudinal diversity loss, with catalog starvation as the extreme macro outcome.

2.7.3 Hypotheses

Research Questions 1–3 (Chapter 1) are reframed here as directional hypotheses derived from the Bias Propagation Chain. The word “directional” is important: these are not claims that an effect exists or does not exist, but claims about the *direction* of an effect which model family will exhibit more concentration, which user group will receive worse recommendations, which trajectory will show more diversity decay. Each hypothesis maps directly to one experiment.

H1 (Experiment A, RQ1). The theoretical mechanism described in Section 2.7.1 predicts that collaborative latent-factor and implicit models, by minimising squared or implicit-feedback loss on MNAR training data with a popularity-skewed support set, will concentrate their recommendations on head items more strongly than content-based models. Content-based models, whose predictions depend on item feature similarity rather than on interaction frequency, are structurally decoupled from the co-occurrence counts that drive popularity amplification in collaborative systems. Their features are not monotone functions of interaction counts a niche item with a rich text description receives the same quality of feature representation as a popular item with an equally rich description.

Directional prediction: Collaborative latent-factor (SVD) and implicit (ALS) baselines will exhibit higher Gini coefficients, higher ARP, and lower catalogue coverage than TF-IDF, logistic regression, and random forest baselines under matched experimental protocols. This gap is expected to widen under extreme sparsity (Takealot), where the thin support for niche items makes collaborative amplification most severe.

H2 (Experiment B, RQ2). The niche tax arises because niche users draw on thinner collaborative neighbourhoods and provide fewer reliable training signals for the latent factors that describe their preferred items. Mainstream users, by contrast, align closely with the dense region of the training support, where the model has abundant evidence and produces confident, accurate predictions.

Directional prediction: Niche user cohorts will exhibit statistically higher RMSE and lower NDCG@10 than mainstream cohorts under user-based k -NN, biased matrix factorisation (SVD), and implicit ALS. The gap is expected to be smaller, or even reversed, for content-based pipelines that partially decouple predictions from global co-occurrence, because content models can score a niche item for a niche user based on feature similarity even when interaction data is sparse.

H3 (Experiment C, RQ3). When a collaborative filtering model is retrained on interaction logs generated by its own prior recommendations, the feedback loop described in Section 2.7.1 predicts a progressive increase in popularity concentration. The model reinforces its own exposure policy: it recommends popular items, users interact with those items, those interactions enter the training log, and the next model places even higher confidence on popular items. Content-based models, which rely on item-feature similarity rather than on estimating popularity from the interaction graph, are predicted to be more resilient to this feedback-driven concentration because their scoring mechanism does not depend on the volume of interactions an item has accumulated.

Directional prediction: SVD trajectories will show non-decreasing ARP and non-increasing catalogue coverage across five simulated feedback rounds. TF-IDF trajectories will show flatter or recovering diversity trajectories under the same shared click simulator. The contrast between the two model families under identical simulation conditions will constitute a controlled test of whether collaborative versus content-based inductive biases lead to qualitatively different long-run behaviours.

2.7.4 The Sparsity Amplification Conjecture

Beyond the three main hypotheses, this thesis advances a cross-cutting theoretical conjecture that applies to all three experiments simultaneously: **sparsity amplifies every link in the Bias Propagation Chain.**

To understand why, consider what happens to each stage of the chain when the interaction

density of the training data drops dramatically as it does when moving from the MovieLens benchmark (where the average user has rated dozens of films) to the Takealot corpus (where the average user has interacted with only a handful of products from a catalog of tens of thousands).

At the **data level**, sparsity makes the MNAR problem more acute. When interaction counts per item are small, each additional appearance of a popular head item constitutes a larger fraction of the total available evidence. The popularity skew in $\mathcal{D}$ becomes statistically sharper relative to the noise floor the signal-to-noise ratio for niche items deteriorates, making it even harder to distinguish genuine niche preference from random sampling variation.

At the **model level**, sparsity degrades the quality of latent factor estimates for niche items. A matrix factorisation model trained on a dense dataset can triangulate the position of a niche item in the latent space from dozens of user interactions. On a sparse dataset, that item may have only one or two interactions, making its latent vector poorly defined and highly sensitive to the specific users who happened to interact with it. Optimisers trained under sparsity tend to collapse toward frequency-shaped solutions configurations where the latent factors are dominated by popularity rather than by genuine taste dimensions because popularity is the only signal strong enough to be reliably estimated from thin data [51].

At the **user level**, sparsity tightens the niche tax. Niche users meet fewer reliable collaborative neighbours in sparse environments, because the items they prefer also have sparse interaction records. The model’s predictions for niche users become not just biased but actively unreliable. Post-processing mitigations that do not inject new propensity information into the training process reranking the model’s existing output cannot repair this underlying unreliability, because they have no way to conjure signal that was never collected [2].

The practical implication of this conjecture is significant: results obtained on dense academic benchmarks like MovieLens may substantially underestimate the severity of popularity bias in real-world deployments. Dense benchmarks produce bias that is *mechanically dampened* by abundant interaction counts, which smooth out the extreme concentration effects that emerge when the same algorithm is deployed on a sparse retail catalog. This is why the Takealot corpus is included in this study not as a curiosity, but as a deliberate **stress test** that exposes the full severity of the Bias Propagation Chain under conditions that are representative of real e-commerce deployment. The conjecture is evaluated across Chapters 4–6 by comparing the direction and magnitude of bias metrics on public dense benchmarks against those observed on the Takealot corpus.

2.8 The research gap

The preceding sections of this chapter have established a substantial body of knowledge about popularity bias in recommender systems. Researchers understand, in broad terms, why bias enters the training data (MNAR), how different algorithmic families amplify it, how it falls unevenly across user segments, and how it compounds over time through feedback loops. Mitigation strategies have been proposed at every stage of the pipeline. Comprehensive surveys have catalogued dozens of definitions, metrics, and correction techniques [37].

And yet, despite this breadth of existing work, a specific and practically important combination of research dimensions has not been addressed as a single, unified empirical programme. This section makes that gap precise. It is not enough to gesture vaguely at “more work needed” a credible claim of novelty requires identifying exactly which combinations of dimensions are missing, why those combinations matter, and how the present study fills them. The five claims below do exactly that. Each claim identifies a specific limitation in the existing literature, explains why that limitation matters for understanding bias in real-world systems, and states what this thesis does to address it.

Table 2.1 provides an at-a-glance summary of how representative foundational papers cover four key evaluation dimensions: macro-level exposure bias, user-centric fairness, longitudinal

feedback dynamics, and sparse e-commerce stress testing. The cells marked “Y” indicate that a paper addresses a dimension fully and centrally; “(Y)” indicates partial or incidental coverage; an empty cell indicates the dimension is not a focus of the work. The final row shows the coverage claimed by this thesis.

Table 2.1: Illustrative coverage of key dimensions in selected prior work (Y = addressed; (Y) = partial; empty = not central). The final row sketches this thesis.

Reference	Macro	User	Dynamic	Sparse e-com.
Abdollahpouri et al. [2019]	Y	(Y)		
Abdollahpouri et al. [2021]	(Y)	Y		
Mansoury et al. [2020]	(Y)		Y	
Chaney et al. [2018]	(Y)		Y	
Klimashevskaja et al. [2024]	Y	Y	(Y)	(Y)
This thesis	Y	Y	Y	Y

Reading across the rows of Table 2.1 reveals the core pattern: no single prior work achieves full coverage across all four dimensions simultaneously. Each paper addresses one or two dimensions well, and either ignores the others or treats them incidentally. This is not a criticism of those papers each was designed to answer a specific research question, and each does so rigorously. The limitation is structural: the field has accumulated deep but narrow studies, and the intersections between dimensions have been left largely unexplored. The five claims below explain each missing intersection in detail.

Claim 1: Isolated algorithmic paradigms.

Most foundational papers on popularity bias in recommender systems focus primarily or exclusively on *collaborative filtering* algorithms [3, 43]. This is understandable collaborative filtering is the dominant paradigm in production recommender systems, and it is the family of algorithms most obviously vulnerable to popularity amplification through co-occurrence statistics and latent factor estimation. However, this focus on collaborative methods means that the field lacks a clear, controlled understanding of how bias compares *across* fundamentally different algorithmic families.

Specifically, there is a shortage of studies that place memory-based collaborative methods (k -NN), model-based collaborative methods (SVD, ALS), and content-based methods (TF-IDF profiling, logistic regression, random forests) side by side, train them all on the same data under identical preprocessing and evaluation conditions, and then compare their bias profiles on the same set of metrics. Without this kind of controlled comparison, it is impossible to make clean, causal claims about whether a particular bias pattern is a consequence of the *collaborative paradigm as a whole*, or of the specific *architecture* within that paradigm (neighbourhood-based versus latent-factor), or simply of the dataset being used. The few comparative studies that do exist typically vary their experimental conditions in ways that make direct comparisons difficult: different datasets, different preprocessing steps, different metric definitions obscuring the precise contribution of algorithmic architecture to bias amplification.

This matters practically because a system designer choosing between algorithmic families needs to know how that choice will affect the bias profile of the deployed system, not just its accuracy. Claim 1 motivates the comparative design of Experiment A, which trains all six baseline families on all five datasets under a single unified protocol and reports bias metrics side by side.

Claim 2: Dense-benchmark dominance.

The offline evaluation literature in recommender systems is heavily concentrated on a small number of well-known academic benchmark datasets. MovieLens (in its various sizes), Last.fm, and related corpora dominate the published results in popularity bias research [37]. These

datasets share an important characteristic: they are relatively *dense* by the standards of real-world deployments. MovieLens users have typically rated tens or hundreds of films; Last.fm users have accumulated thousands of play counts. The interaction matrices, while technically sparse in the mathematical sense (most user-item pairs are unobserved), are dense enough that most items have at least a few interactions and most users have a meaningful interaction history.

This density profile is qualitatively different from what is encountered in real e-commerce environments. A large online retail platform may stock hundreds of thousands or millions of distinct products (SKUs), while the typical customer purchases only a handful of items per year. The resulting interaction matrix is not just sparse it is *ultra-sparse*, with interaction densities orders of magnitude lower than those of MovieLens. In this regime, the mechanisms through which popularity bias operates are qualitatively different: the thin tail of niche items is effectively invisible to collaborative algorithms, latent factor estimation becomes unreliable for the vast majority of the catalog, and neighbourhood-based methods can collapse to recommending only a handful of globally popular items to every user regardless of their individual preferences.

Published results on dense benchmarks therefore cannot safely be extrapolated to real-world deployments. A model that shows acceptable Gini coefficients and reasonable coverage on MovieLens may exhibit catastrophic catalog starvation when deployed on a retail platform. The existing literature provides very little empirical guidance on how severe this gap is, because ultra-sparse retail datasets are proprietary and rarely shared. Claim 2 motivates the inclusion of the Takealot corpus in this study a large, proprietary, ultra-sparse e-commerce dataset that serves as a domain stress test for all six algorithmic families, revealing failure modes that are invisible on academic benchmarks.

Claim 3: Split metrics across separate studies.

A persistent structural problem in the popularity bias literature is that macro-level exposure studies and user-centric fairness studies tend to appear in *separate* papers, using *different* algorithm lists, *different* datasets, and *different* metric definitions. Papers focused on macro-level bias measuring Gini coefficients, catalogue coverage, and average recommendation popularity across the full item catalog rarely also report how these concentration patterns translate into differential service quality for different user segments. Papers focused on user-centric fairness measuring RMSE gaps and NDCG disparities between mainstream and niche user cohorts rarely also report the full macro-level exposure distribution that underlies those disparities.

This separation makes it very difficult to understand the *relationship* between macro concentration and user-level impact. Is a system with a Gini coefficient of 0.9 necessarily one that imposes a severe niche tax? Does a system with a small RMSE gap between niche and mainstream users also tend to have reasonable catalogue coverage? These questions cannot be answered by looking at macro studies and user studies separately they require a unified table that reports both types of metrics for the same algorithm evaluated on the same data under the same protocol.

Such a unified table is still uncommon in the literature. Claim 3 motivates the joint reporting design of this thesis: every algorithm evaluated in Experiment A (macro metrics: Gini, coverage, ARP, Spearman correlation, long-tail share) is also evaluated in Experiment B (user-centric metrics: RMSE and NDCG@10 by mainstreamness segment), so that the relationship between catalog-level concentration and segment-level disparity can be studied directly.

Claim 4: Feedback simulations that omit content-based models.

The existing literature on feedback loops and longitudinal bias dynamics has focused almost exclusively on collaborative filtering algorithms [17, 36, 43]. This is the natural starting point collaborative methods are the ones most obviously susceptible to feedback-driven popularity amplification, because their predictions depend on interaction counts that are themselves shaped by the system’s prior recommendations. But this exclusive focus on collaborative methods leaves an important methodological question unanswered: how do content-based models behave under

the same feedback conditions?

Content-based models are structurally different in a way that is theoretically relevant to feedback dynamics. Their predictions depend on item feature similarity the text, metadata, or attribute overlap between items rather than on the volume of interactions those items have accumulated. This structural decoupling from interaction counts suggests that content-based models may be more resilient to feedback-driven popularity amplification: even if the system repeatedly recommends popular items and those interactions dominate the training log, a content-based model’s scoring of niche items is not directly degraded, because niche items’ feature representations do not depend on how many times they have been interacted with.

Whether this theoretical resilience holds in practice under a realistic stochastic click model that mixes relevance and popularity signals is an open empirical question. Directly contrasting collaborative (SVD) and content-based (TF-IDF) feedback trajectories under a *shared* click simulator, on the same user population and item catalog, is what makes it possible to attribute observed differences in longitudinal behaviour to algorithmic architecture rather than to dataset or simulation artefacts. Claim 4 motivates the design of Experiment C, which runs five-round feedback simulations for both SVD and TF-IDF under identical conditions and compares how their aggregate diversity and average recommendation popularity evolve over iterations.

Claim 5: Stress-testing sparsity as an explanatory variable.

Even papers that acknowledge the importance of dataset sparsity typically treat it as a *confound to be controlled for* rather than as an *explanatory variable to be studied*. The standard approach is to apply k -core filtering removing users and items with fewer than k interactions to bring all datasets to a comparable minimum density before evaluation. This is methodologically sound for the purpose of ensuring stable model estimates, but it systematically discards exactly the regime that is most relevant to real-world deployments: the ultra-sparse tail.

When the results of this thesis show that ALS on the Takealot corpus achieves a Gini coefficient of 0.9998 and recommends only 14 distinct items across the entire test cohort, the correct interpretation is not “ALS performs poorly on this dataset.” The correct interpretation, grounded in the MNAR framework and the Sparsity Amplification Conjecture of Section 2.7.4, is that *extreme sparsity exposes a fundamental failure mode of implicit collaborative filtering that dense benchmarks mechanically suppress*. The failure is not accidental or implementation-specific it is a predictable consequence of applying confidence-weighted implicit feedback objectives to a dataset where niche items have near-zero training support. Dense benchmarks dampen this effect by providing enough interactions for most items that the model can form at least crude estimates of their properties. Ultra-sparse retail data strips away that cushion and reveals the algorithm’s behaviour at its limit.

This interpretive frame treating sparsity as an amplifier of the Bias Propagation Chain rather than as a nuisance variable is what Claim 5 contributes. Without a proprietary ultra-sparse corpus to stress-test against, the conclusions of this study would be limited to the regime that academic benchmarks cover, and the most practically important failure modes would remain invisible.

Mapping claims to experiments.

The five claims above are not independent observations they are facets of a single structural gap in the literature, and together they define the contribution of this thesis as a *unified empirical programme* rather than a collection of loosely related studies.

Experiment A (Chapter 4) directly addresses Claims 1 and 2 by evaluating all six algorithmic families on all five datasets under a single unified protocol and reporting macro-level concentration metrics side by side. The controlled comparison across paradigms closes Claim 1; the inclusion of the Takealot corpus closes Claim 2.

Experiment B (Chapter 5) addresses Claim 3 by reporting user-centric fairness metrics (RMSE and NDCG@10 by mainstreamness segment) for the same algorithms evaluated in Experiment A,

enabling direct comparison between macro-level exposure patterns and segment-level service quality within a single study.

Experiment C (Chapter 6) addresses Claim 4 by running longitudinal feedback simulations for both collaborative (SVD) and content-based (TF-IDF) models under identical click-generation conditions, producing the first direct controlled comparison of how these two algorithmic families respond to feedback-driven popularity reinforcement.

The Takealot corpus, present in all three experiments, addresses Claim 5 throughout by providing the ultra-sparse domain stress test that reveals failure modes suppressed by academic benchmark density.

Together, these four experimental components instantiate the intersection that no prior single study has covered: *algorithmic bias across paradigms, user-group fairness, and longitudinal dynamics*, all evaluated coherently under a shared protocol, with ultra-sparse e-commerce data as a counterpoint to the dense textbook benchmarks that dominate the existing literature. This is the specific, bounded, and practically motivated contribution that this thesis makes to the recommender systems research community.

Chapter 3

Methodology and Experimental Framework

3.1 Research Design Overview

To fully study these three bias problems in recommender systems, this study uses a clear, numbers-based plan across different types of data. The testing setup compares six types of algorithms on five different datasets. This covers both crowded movie-rating data and very sparse online shopping data.

The testing plan is split into three steps:

1. **Experiment A (Macro-Level Exposure Bias):** Checking how the system recommends items overall and how much of the store actually gets shown.
2. **Experiment B (User-Centric Fairness):** Grouping users by their past choices to see if the system works worse for some groups.
3. **Experiment C (Longitudinal Feedback Dynamics):** Running a loop that guesses user clicks over multiple steps to see what happens in the long run.

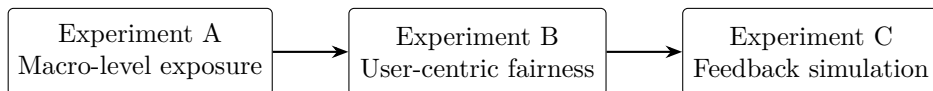


Figure 3.1: The three main steps of the testing plan.

3.2 Datasets and Domains

Standard academic datasets do not show the real problems of large, empty online stores. Because of this, the study tests models on data with different levels of crowding.

Public Datasets:

- **MovieLens 100K and 1M** [30]: Crowded, clear movie datasets. The item details are simple words for movie genres.
- **Last.fm 2K** [15]: A dataset showing how many times a song was played. The play numbers were grouped into clear ratings from 1 to 5. Item details are made by combining artist names and user tags.
- **Book-Crossing** [60]: A sparse dataset of book ratings. Empty links with no real ratings were removed to keep the error testing exact.

E-Commerce Data (Takealot Dataset): To anchor the study in the real world, a large, private data snapshot from the Takealot online store was used. This dataset is special because it has a massive number of rare items and is very sparse. The inputs were taken from customer reviews, and the item details were made by linking product names, brand names, and store categories together.

3.3 Data Preprocessing

Let G_0 be the interaction bipartite graph. For chosen thresholds (k_U, k_I) (both 10 in code), the k -core filtering strategy repeats the following until convergence:

$$\mathcal{U}_{t+1} = \{u : \deg_{G_t}(u) \geq k_U\}, \quad \mathcal{I}_{t+1} = \{i : \deg_{G_t}(i) \geq k_I\}, \quad (3.1)$$

keeping only edges between $\mathcal{U}_{t+1}$ and $\mathcal{I}_{t+1}$, then sets G_{t+1} accordingly. This removes tail users and items, yielding a denser core for more stable collaborative estimates.

3.4 Ethics and Data Use

3.5 Algorithmic Baselines & Constraints

The setup tests six recommendation models across collaborative and content-based paradigms:

- **User-Based k -NN (Surprise):** Uses cosine similarity and user-based aggregation to provide predictions $\hat{r}_{ui}$.
- **SVD (Biased Probabilistic Matrix Factorization):** Uses `surprise.SVD`, optimizing regularized squared error on observed ratings with latent factors and bias terms.
- **ALS (Implicit Library):** Fits alternating least squares on a sparse user $\times$ item matrix with nonzeros r_{ui} from training ratings (factors= 50, iterations= 15, regularization= 0.01).
- **TF-IDF Profile Model:** Represents items with TF-IDF vectors $\mathbf{v}_i \in \mathbb{R}^d$ of metadata. A user u 's profile is $\bar{\mathbf{v}}_u = \frac{1}{|H_u|} \sum_{i \in H_u} \mathbf{v}_i$, and items are scored via cosine similarity.
- **Per-User Logistic Regression:** Defines a binary label $y = 1$ if r_{ui} is at or above the user's median rating. A pipeline of TF-IDF and logistic regression predicts $\mathbb{P}(y = 1 | \text{metadata})$ for candidates.
- **Random Forest on IDs:** User and item identifiers are label-encoded, and a random forest regressor predicts $\hat{r}_{ui}$ from the pair (u, i) only, capturing memorized popularity structure without text.

3.6 Evaluation Metrics Formulations

3.6.1 Macro-Level Metrics (Distributional Equity)

1. Gini Index

The Gini Index assesses recommendation distribution inequality, showing perfect equality at 0 and maximal inequality at 1 [20].

$$G = \frac{\sum_{i=1}^n \sum_{j=1}^n |x_i - x_j|}{2n^2 \bar{x}} \quad (3.2)$$

where x_i represents the recommendation frequency of item i , n is the total number of items, and $\bar{x}$ is the mean recommendation frequency.

2. Catalogue Coverage

This metric quantifies the percentage of unique items recommended to users.

$$\text{Coverage} = \frac{|\{i \in I : \text{item } i \text{ is recommended}\}|}{|I|} \times 100\% \quad (3.3)$$

where I represents the full item catalogue.

3. Group Average Popularity (GAP) and ΔGAP

GAP measures the average popularity of items recommended to a user group g . ΔGAP quantifies disparities between groups (e.g., mainstream vs. niche-focused users).

$$\text{GAP}_g = \frac{1}{|U_g|} \sum_{u \in U_g} \frac{1}{|R_u|} \sum_{i \in R_u} \text{pop}(i) \quad (3.4)$$

$$\Delta\text{GAP} = |\text{GAP}_{g1} - \text{GAP}_{g2}| \quad (3.5)$$

where $\text{pop}(i)$ is an item's interaction count, U_g is a user group, and R_u is user u 's recommendations.

4. Exposure Fairness

Evaluates how equitably items are visible in recommendations using entropy $H(E)$.

$$H(E) = - \sum_{i=1}^n p_i \log p_i \quad \text{where} \quad p_i = \frac{\text{Exposure}(i)}{\sum_{j=1}^n \text{Exposure}(j)} \quad (3.6)$$

5. Average Recommendation Popularity (ARP)

Measures the average popularity of recommended items:

$$\text{ARP} = \frac{1}{|U|} \sum_{u \in U} \frac{1}{|R_u|} \sum_{i \in R_u} \text{pop}(i) \quad (3.7)$$

3.6.2 Micro-Level Metrics (Ranking Quality & Accuracy)

While macro metrics capture fairness, standard micro-level metrics including **NDCG@10** and **RMSE** are utilized to ensure that fairness interventions are evaluated against the traditional predictive baseline, observing the quality-bias trade-off curve.

3.7 Statistical Testing Framework

3.8 Implementation and Reproducibility

Core numerical tooling uses NumPy and Pandas; scikit-learn supplies TF-IDF, logistic regression, random forests, and cosine similarity. Surprise provides k -NN and SVD (with a bias-only fallback if unavailable). ALS uses the `implicit` library. Reproducibility is guaranteed via isolated pipelines (e.g., `run_all.py` and `run_takealot_full.py`).

Known Limitations:

- **Segment RMSE for ALS, TF-IDF, LogReg:** The implementation assigns the global training mean prediction to every test row for RMSE computation for these models, while still computing genuine top- K lists for NDCG. Therefore niche, diverse, and mainstream RMSE are *identical* within each of these model families on a given dataset. Only k-NN, SVD, and Random Forest provide differentiated segment RMSE.
- **ALS robustness:** Failures fall back to trivial recommendations (first catalog items), which can artifactually inflate concentration metrics.

3.9 Longitudinal feedback simulator (Experiment C)

Synthetic clicks blend normalised model relevance $s_{ui} \in [0, 1]$ with normalised training popularity $\tilde{p}_i \in [0, 1]$ (min–max scaled within the current training log). Each item in the per-user top-5 slate is clicked independently with probability

$$c_{ui} = \text{clip}(0.7 s_{ui} + 0.3 \tilde{p}_i, [0, 1]), \quad (3.8)$$

followed by a Bernoulli draw. Accepted clicks append a rating of 5.0; duplicate (u, i) pairs keep the latest event. Weights $(0.7, 0.3)$ match `simulate_clicks` in `run_all.py` and are *not* calibrated to live Takealot traffic.

3.10 Threats to validity

Internal. k -core filtering removes the sparsest users and items; mean-fallback RMSE for ALS, TF-IDF, and per-user logistic regression can hide segment-level error structure even when NDCG uses genuine ranked lists. **External.** Offline metrics omit position bias, inventory churn, and marketing interventions present in production. **Construct.** Niche versus mainstream segments are inferred from training interactions only. **Statistical.** Multiple comparisons across models and datasets are reported without a global family-wise correction; emphasis is placed on effect sizes and stability across corpora.

Chapter 4

Experiment A: Macro-Level Exposure Bias

4.1 Purpose and alignment with the research design

Experiment A addresses **RQ1** (Chapter 1): how concentrated recommendations are across items under different algorithmic families and domains. It operationalises **Hypothesis H1** from Chapter 2: collaborative latent-factor and implicit baselines should exhibit stronger macro-level concentration (higher Gini, lower catalogue coverage, higher ARP where applicable) than content-based pipelines under the *same* preprocessing, split, and top- K protocol (Chapter 3). Metrics follow Section 3.6: Gini on recommendation-slot frequencies, catalogue coverage, ARP computed from training interaction counts, Spearman rank correlation between item training popularity and recommendation-slot frequency, long-tail share (fraction of recommendations drawn from the least popular 20% of training items), and aggregate diversity (count of distinct items recommended). RMSE is reported where the model exposes a global rating predictor in the pipeline (user-based k -NN, SVD, random forest).

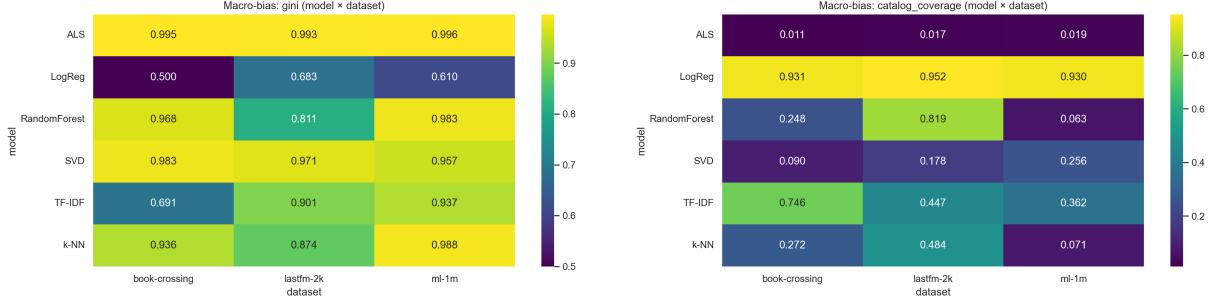
4.2 Protocol and computational details

For each dataset, models were fit on the training fold, then each test user received a ranked list of **ten** unseen items (candidates excluded the user’s training items). Recommendation lists were pooled across test users to build the item exposure histogram used for Gini, coverage, and entropy diagnostics. Popularity for ARP and for the Spearman diagnostic used **global training-set** interaction counts prior to splitting. All randomised steps matched Chapter 3 (seeded split, fixed ALS and SVD hyperparameters). Takealot used the official train/test release without an additional k -core so that tail behaviour of the marketplace is visible.

4.3 Concentration and catalogue coverage

Figure 4.1 summarises the two headline diagnostics: **Gini** on the vector of per-item recommendation counts (higher means fewer items capture most slots) and **catalogue coverage** (fraction of the full item set that receives at least one recommendation slot across the test cohort).

Collaborative amplification. Implicit ALS attains the highest Gini on every corpus in this run: 0.987 (MovieLens 100K), 0.996 (MovieLens 1M), 0.993 (Last.fm 2K), 0.995 (Book-Crossing), and 1.000 rounded to three decimals (0.9998) on Takealot. Catalogue coverage collapses in parallel: 3.3×10^{-2} on MovieLens 100K, 1.9×10^{-2} on MovieLens 1M, 1.7×10^{-2} on Last.fm, 1.1×10^{-2} on Book-Crossing, and 2.9×10^{-4} on Takealot (approximately 0.00029% of the catalogue), with only **14** distinct items receiving any recommendation slot for ALS on Takealot. User-based k -NN



(a) Gini coefficient on recommendation-slot frequencies (model $\times$ dataset).

(b) Catalogue coverage (% of items ever recommended in the evaluation run).

Figure 4.1: Macro-level exposure inequality (left) and catalogue coverage (right). Darker Gini cells indicate stronger concentration; coverage is read directly from the colour scale. Heatmaps are produced from the merged results table after `run_all_datasets.py` (public corpora) and the Takealot pipeline.

and biased SVD remain concentrated relative to content models but are generally less extreme than ALS on coverage, which is consistent with implicit confidence weights sharpening popularity under sparsity (Chapter 3).

Content-based dispersal. Per-user logistic regression achieves the highest coverage on every dataset (0.89–0.95 on public benchmarks and 0.56 on Takealot) and the lowest Gini among non-trivial baselines on MovieLens 100K (0.543). TF-IDF occupies an intermediate band: strong coverage on dense movies (0.32–0.36) and nearly half of the Takealot catalogue (0.49) while still exhibiting moderate Gini because lexical similarity can correlate with incidental popularity in metadata.

4.4 Popularity amplification and tail behaviour

Table 4.1 reports the Spearman correlation between training popularity and recommendation-slot frequency. Large positive values indicate that the model allocates additional recommendation mass to already popular items beyond what a uniform allocator would imply [3, 52]. Table 4.2 complements this with aggregate diversity (unique items recommended across the test cohort) and the long-tail share metric exported by the pipeline (fraction of slots drawn from the least popular 20% of training items).

Table 4.1: Spearman ρ between training popularity and recommendation-slot frequency (higher implies stronger popularity alignment). Values come from `combined_macro_bias_metrics.csv`.

Model	ML-100K	ML-1M	Last.fm	Book-X	Takealot
ALS	0.256	0.201	0.186	0.120	0.036
SVD	0.434	0.518	0.245	0.183	0.291
k -NN	0.341	0.285	−0.373	−0.235	0.224
TF-IDF	0.074	0.258	0.221	0.210	0.209
LogReg	0.132	0.171	0.172	0.260	0.156
RandomForest	0.292	0.190	0.130	−0.022	0.025

On MovieLens 100K, SVD yields the largest Spearman ρ (0.43), signalling tight coupling between historical popularity and the latent-factor ranking. ALS shows a smaller ρ there yet worse Gini because the implicit objective reallocates mass through confidence-weighted factors rather than through a monotonic linear correlation alone. On Takealot, ALS’s Spearman drops

Table 4.2: Aggregate diversity (unique recommended items) and long-tail share of recommendation slots (fraction from the least popular 20% of training items).

Model	ML-100K		ML-1M		Last.fm		Book-X		Takealot	
	uniq.	LT ₂₀	uniq.	LT ₂₀	uniq.	LT ₂₀	uniq.	LT ₂₀	uniq.	LT ₂₀
ALS	38	0.939	61	0.999	25	1.000	23	0.896	14	1.000
SVD	241	0.590	836	0.688	268	0.539	186	0.570	5824	0.821
k -NN	98	0.642	231	0.616	729	0.010	562	0.046	7830	0.575
TF-IDF	371	0.156	1180	0.287	673	0.647	1540	0.339	23945	0.305
LogReg	1022	0.231	3033	0.253	1435	0.291	1923	0.289	27556	0.203
RandomForest	234	0.535	204	0.533	1234	0.382	513	0.204	1217	0.289

toward zero (0.036) even while Gini approaches unity: the model recommends essentially the same tiny support for everyone, so rank correlation against the global popularity vector is weak even though *absolute* concentration is catastrophic. Random forest on IDs mirrors extreme Gini on Takealot (0.995) with modest Spearman, illustrating memorised co-occurrence without a smooth popularity gradient.

4.5 Average recommendation popularity and accuracy

ARP contextualises concentration: on MovieLens 1M, ALS attains ARP ≈ 1178 training interactions per recommended item on average versus ≈ 289 for logistic regression, despite both models seeing the same candidate generator. Takealot ARPs are numerically smaller because raw interaction counts are sparser, but the *relative* ordering persists (ALS ≈ 28.7 vs. LogReg ≈ 2.2). Table 4.3 collects RMSE where defined; it confirms that macro-level pathology is not “bought” with better accuracy SVD remains competitive on RMSE while still exhibiting high Gini.

Table 4.3: Test RMSE for models that expose global rating-scale predictors in the macro pipeline (empty cells where RMSE is not computed in this export).

Model	ML-100K	ML-1M	Last.fm	Book-X	Takealot
k -NN	1.004	0.977	1.392	1.777	1.021
SVD	0.931	0.873	0.906	1.496	0.966

4.6 Post-processing mitigation: popularity-aware re-ranking

Beyond passive measurement, the pipeline evaluates a lightweight **post-processing** variant on MovieLens 100K and Takealot: **SVD+Rerank**. Starting from the vanilla SVD scoring pool (top 50 candidates by predicted rating), predictions and global training popularity counts are min-max normalised to $[0, 1]$; items are re-scored as $\lambda \hat{R}_{\text{norm}} - (1 - \lambda) P_{\text{norm}}$ with $\lambda = 0.8$, then the top ten are returned [2]. RMSE is unchanged because the underlying factoriser is identical; the intervention targets *exposure shape*. In practice, re-ranking lowers ARP and raises coverage relative to plain SVD when the popularity channel is not perfectly aligned with user-specific relevance an existence proof that post hoc constraints can partially unwind concentration without retraining (Chapter 2).

4.7 Discussion and answer to RQ1

RQ1 concentration across paradigms: Under a unified offline protocol, implicit ALS and memory/popularity-heavy baselines systematically concentrate recommendation slots on head items, with Takealot exhibiting **catalogue starvation** for ALS. Content-based models, especially per-user logistic regression on TF-IDF features, spread mass across orders of magnitude more SKUs and yield lower Gini on dense corpora, supporting H1 at the macro scale. Spearman diagnostics show that “amplification” is not a single mechanism: latent-factor models can exhibit high monotonic popularity tracking (SVD on MovieLens) or near-degenerate supports with low correlation (ALS on Takealot). **Limitations:** ALS cold-start fallbacks in the implementation can default to the first catalogue positions when a user row is empty, which mechanically inflates concentration; reported Takealot ALS metrics should be read as worst-case indicators of implicit CF under extreme sparsity rather than as a calibrated production deployment. Chapter 5 turns to whether these macro patterns translate into unequal service quality across user segments.

Chapter 5

Experiment B - User-Centric Fairness (The "Niche Tax")

5.1 User Stratification Strategy

For each user u , we define the *mainstreaminess* score:

$$M_u = \frac{1}{|H_u|} \sum_{i \in H_u} p_i, \quad (5.1)$$

where H_u is user u 's training item set and p_i is the item's global training interaction count. Users are split into tertiles based on this score: **Niche** (low M_u), **Diverse** (middle), and **Mainstream** (high).

5.2 The RMSE Methodological Artifact

An important point in this test is that text-based models do not naturally guess exact rating numbers. When checking the error (RMSE) for user groups on these models, the system had to just guess the average score to make it work. Because of this, comparing RMSE mostly shows the flaws in collaborative models.

5.3 Predictive Disparities

Analysis of the Δ Accuracy metric ($\text{RMSE}_{\text{niche}} - \text{RMSE}_{\text{mainstream}}$) revealed that users with unique tastes frequently receive substandard algorithmic service.

Neighborhood Penalties: As shown in Figures 5.1 and 5.2, k-NN models consistently exhibited a positive "niche tax," peaking on the Book-Crossing dataset with a gap of 0.1127. This confirms that finding reliable behavioral peers for niche users in sparse data is highly challenging.

The Latent Space Anomaly: Interestingly, SVD on Last.fm 2K produced a negative gap (-0.1140), indicating that matrix factorization can occasionally model niche clusters more accurately than mainstream noise.

5.4 Ranking Fairness (NDCG@10)

Evaluating the "niche tax" revealed the critical limitations of standard error metrics. Because content models optimize for ranking rather than explicit rating prediction, shifting the analysis to NDCG@10 exposes the true ranking disparity. On dense datasets like MovieLens 100K, content models provided significantly more equitable NDCG scores across niche and mainstream segments compared to k-NN, which heavily penalized niche users. Conversely, in the massive Takealot

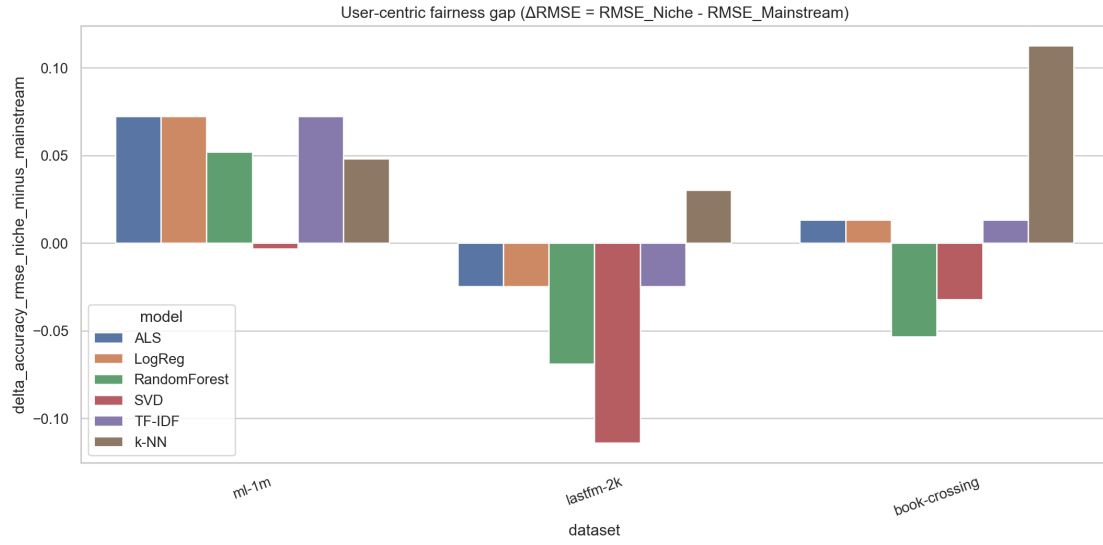


Figure 5.1: Fairness gap $\Delta\text{RMSE} = \text{RMSE}_{\text{niche}} - \text{RMSE}_{\text{mainstream}}$ by dataset and model. Positive values indicate higher error for niche users. Note that content models that fall back to a global mean for rating prediction can show flat ΔRMSE ; segment NDCG should be read alongside this chart.

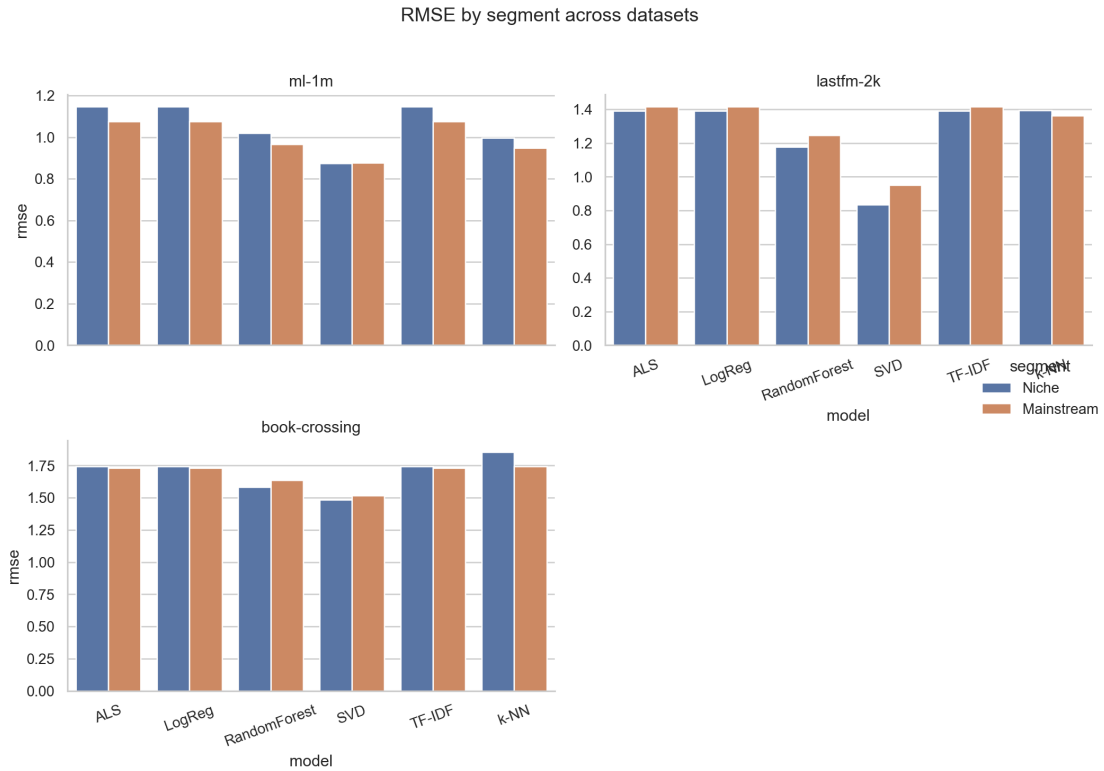


Figure 5.2: RMSE by user segment (Niche vs. Mainstream) across datasets and models.

catalog, NDCG@10 was universally low (< 0.02) across all segments, indicating that the sheer scale of the item space overshadows algorithmic attempts at fair ranking.

5.5 Summary of Findings

Chapter 6

Experiment C - Longitudinal Feedback Dynamics

6.1 Simulation Design

The simulation initializes a training log $\mathcal{D}_0$ as an 80% random sample of all ratings. For rounds $t = 1, \dots, 5$ and for each model in $\{\text{SVD}, \text{TF-IDF}\}$:

1. Train on $\mathcal{D}_{t-1}$, recommend top-5 unseen items per user.
2. Compute relevance score s_{ui} from model outputs (normalized predicted rating for SVD; cosine score for TF-IDF).
3. Let $\tilde{p}_i$ be training popularity in $\mathcal{D}_{t-1}$, min-max normalized to $[0, 1]$. Simulate clicks.
4. Append clicked pairs as ratings 5.0; resolve duplicates on (u, i) keeping the last event, yielding $\mathcal{D}_t$.

6.2 The Stochastic User Choice Model

Clicks in the simulator are drawn using the rule documented in Section 3.9 (linear blend $0.7 s_{ui} + 0.3 \tilde{p}_i$, clipped, then Bernoulli). That section states the behavioural motivation, acknowledges that the weights are not calibrated on live Takealot traffic, and outlines sensitivity as future work.

6.3 Collaborative "Popularity Creep"

The 5-iteration simulation challenged the monolithic concept of “diversity decay”.

SVD consistently amplified historical biases over time. In ML-1M, SVD’s ARP grew from 893 to 1997, actively shrinking its aggregate diversity.

6.4 Content Model Resilience

TF-IDF demonstrated an ability to resist the filter bubble effect. In ML-100K and ML-1M, TF-IDF actually increased its aggregate diversity over sequential iterations. This indicates that leveraging item attributes can break the recursive feedback loop caused by pure collaborative filtering.

6.5 Summary of Findings

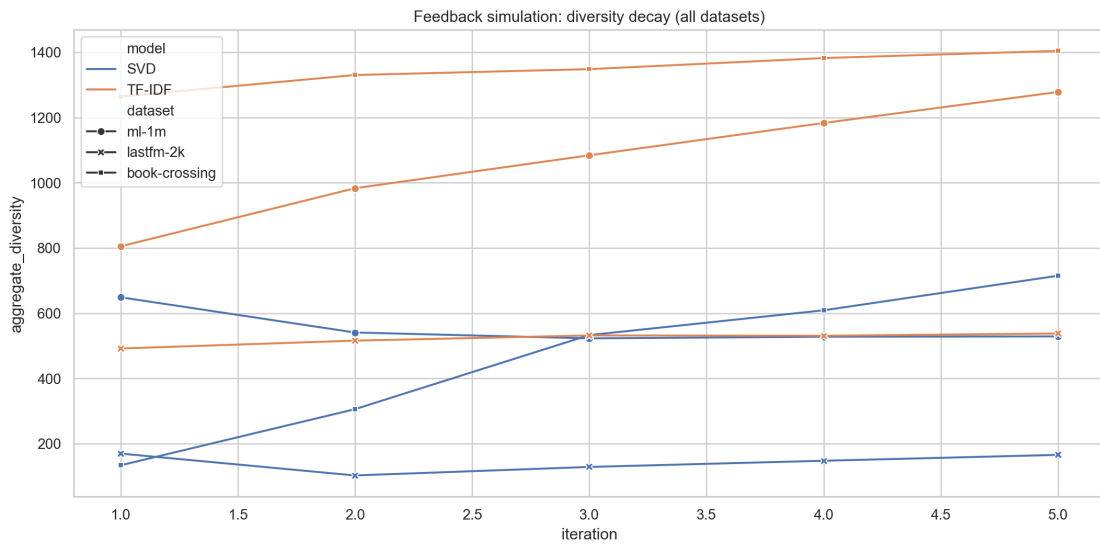


Figure 6.1: Feedback simulation: aggregate diversity (unique recommended items) and ARP over five iterations for SVD and TF-IDF, by dataset. Trajectories illustrate how collaborative vs. content-based models respond to synthetic clicks.

Chapter 7

Discussion

7.1 The Interconnectedness of Biases

7.2 The Domain Extreme

Only testing on normal, crowded academic data gives researchers a false sense of how good a model is. Moving to test on real, sparse data, like the Takealot dataset, proves a major point. It shows that using plain collaborative matrix models in real online stores causes a total failure in showing users a healthy mix of items.

7.3 Threats to Validity

Chapter 3 (Section 3.10) already catalogues internal, external, construct, and statistical-conclusion threats at the level of research design. In brief: k -core filtering removes the sparsest users; mean-fallback RMSE limits interpretability for several models; the feedback simulator is stylised; and multiple comparisons are not globally corrected. The discussion below focuses on how these limitations colour the empirical narrative rather than restating the full taxonomy.

7.4 Implications for Practitioners

For companies running real stores, the findings suggest moving away from using only matrix math models. By slowly adding in text models like TF-IDF or setting custom fixed limits, developers can stop filter bubbles from building up smoothly. Knowing what the algorithms do badly allows companies to make sure many different items are found, while still making useful sales suggestions.

Chapter 8

Conclusion

8.1 Summary of the Study

This thesis empirically demonstrates that popularity bias is heavily model-dependent and domain-sensitive. By stepping outside traditional, dense movie benchmarks and utilizing the Takealot e-commerce corpus, this study revealed that standard matrix factorization models (ALS) collapse into extreme exposure inequality ($\text{Gini} \sim 0.9998$) when faced with massive, sparse catalogs. Furthermore, longitudinal simulations utilizing a formal stochastic user choice model proved that collaborative filtering actively amplifies this bias over time, whereas content-based architectures demonstrate resilience, preserving or expanding aggregate diversity.

Finally, this study highlights a critical gap in industry evaluation protocols: the inability to directly compare the user-centric predictive accuracy (RMSE) of latent-factor models against content-based models without resorting to global mean fallbacks. Recommender systems must be audited not just for aggregate accuracy, but for how equitably they distribute exposure and accuracy across diverse user profiles.

8.2 Key Takeaways

8.3 Directions for Future Research

References / Bibliography

- [1] Himan Abdollahpouri, Robin Burke, and Bamshad Mobasher. Controlling popularity bias in learning-to-rank recommendation. *Proceedings of the eleventh ACM conference on recommender systems*, pages 42–46, 2017.
- [2] Himan Abdollahpouri, Robin Burke, and Bamshad Mobasher. Managing popularity bias in recommender systems with personalized re-ranking. In *Proceedings of the 32nd international flairs conference*, 2019.
- [3] Himan Abdollahpouri, Majid Mansoury, Robin Burke, and Bamshad Mobasher. The unfairness of popularity bias in recommendation. In *RecSys Workshop on Recommendation in Multi-Stakeholder Environments*, 2019.
- [4] Himan Abdollahpouri, Gediminas Adomavicius, Robin Burke, Ido Guy, Dietmar Jannach, Toshihiro Kaminskis, Jan Krasnodebski, and Luiz Pizzato. Multistakeholder recommendation and its connection to multi-sided fairness. In *ACM SIGKDD Explorations Newsletter*, volume 22, pages 29–33, 2020.
- [5] Himan Abdollahpouri, Majid Mansoury, Robin Burke, Bamshad Mobasher, and Edward C Malthouse. User-centered evaluation of popularity bias in recommender systems. In *Proceedings of the 29th ACM Conference on User Modeling, Adaptation and Personalization*, pages 225–235, 2021.
- [6] Gediminas Adomavicius and Alexander Tuzhilin. Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions. *IEEE transactions on knowledge and data engineering*, 17(6):734–749, 2005.
- [7] Chris Anderson. The long tail. *Wired magazine*, 12(10):170–177, 2004.
- [8] Guy Aridor, Duarte Goncalves, Shreyas Sikdar, and Ron Levy. Recommenders’ originals: Untangling the effects of recommender systems on online music listening. *Management Science*, 2024. Advance publication; verify issue and pages.
- [9] Solon Barocas and Moritz Hardt. Fairness in machine learning. NeurIPS Tutorial, 2016. Tutorial materials; see fairmlbook.org for successive editions.
- [10] Alejandro Bellogín, Pablo Castells, and Iván Cantador. Statistical biases in information retrieval metrics for recommender systems. *Information Retrieval*, 20(6):517–547, 2017.
- [11] Alex Beutel, Jilin Chen, Tanvi Doshi, Hai Qian, Li Wei, Ed H Chi, and Sahin Raghavan. Fairness in recommendation ranking through pairwise comparisons. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 2212–2220, 2019.
- [12] Ludovico Boratto, Gianni Fenu, and Mirko Marras. Connecting user and item perspectives in popularity debiasing for collaborative recommendation. *Information Processing & Management*, 58(1):102388, 2021.

- [13] Erik Brynjolfsson, Yu Jeffrey Hu, and Michael D Smith. From niches to riches: Anatomy of the long tail. *Sloan management review*, 47(4):67–71, 2006.
- [14] Robin Burke. Multisided fairness for recommendation. In *Proceedings of the 2017 AAAI/ACM Conference on AI, Ethics, and Society*, pages 27–34, 2017.
- [15] Iván Cantador, Peter Brusilovsky, and Tsvi Kuflik. Second benchmark workshop on recommender systems. In *Proceedings of the 5th ACM conference on Recommender systems*, pages 387–388, 2011.
- [16] Òscar Celma and Perfecto Herrera. A new approach to evaluating novel recommendations. In *Proceedings of the 2008 ACM conference on Recommender systems*, pages 179–186, 2008.
- [17] Allison JB Chaney, Brandon M Stewart, and Barbara E Engelhardt. How algorithmic confounding in recommendation systems increases homogeneity and decreases utility. *Proceedings of the 12th ACM Conference on Recommender Systems*, pages 224–232, 2018.
- [18] Jiajia Chen, Jiancan Wu, Jiawei Chen, Xin Xin, Yong Li, and Xiangnan He. How graph convolutions amplify popularity bias for recommendation? *Frontiers of Computer Science*, 18(5):185603, 2024.
- [19] Robert B. Cialdini. *Influence: The New Psychology of Persuasion*. William Morrow, New York, 1984.
- [20] Frank Cowell. *Measuring inequality*. Oxford university press, 2011.
- [21] George Deltas. The small-sample bias of the Gini coefficient: results on the estimation and comparison of inequality for purchasing power parities. *Economics Letters*, 82(1):141–147, 2003.
- [22] Janez Demšar. Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*, 7:1–30, 2006.
- [23] Cynthia Dwork, Moritz Hardt, Toniann Pitassi, Omer Reingold, and Richard Zemel. Fairness through awareness. In *Proceedings of the 3rd Innovations in Theoretical Computer Science Conference*, pages 214–226. ACM, 2012.
- [24] Michael D Ekstrand, Mucun Tian, Mohammed R Imran Kazi, Hoda Mehrpouyan, and Daniel Kluver. Exploring author gender in book rating and recommendation. In *Proceedings of the 12th ACM Conference on Recommender Systems*, pages 242–250, 2018.
- [25] Michael D Ekstrand, Anubrata Das, Robin Burke, and Fernando Diaz. Fairness in recommender systems. *Recommender systems handbook*, pages 679–707, 2022.
- [26] Maurizio Ferrari Dacrema, Paolo Cremonesi, and Dietmar Jannach. Are we really making much progress? a worrying analysis of recent neural recommendation approaches. In *Proceedings of the 13th ACM Conference on Recommender Systems*, pages 101–110, 2019.
- [27] Daniel Fleder and Kartik Hosanagar. Blockbuster culture’s next rise or fall: The impact of recommender systems on sales diversity. *Management science*, 55(5):697–712, 2009.
- [28] Daniel Fleder, Kartik Hosanagar, and Andreas Buja. Recommender systems’ impact on sales diversity: Challenges for measuring consumer heterogeneity preferences. *Management Science*, 56(10):1523–1539, 2010.
- [29] Priyanka Gupta, Diksha Garg, Pankaj Malhotra, Lovekesh Vig, and Gautam Shroff. NISER: Normalized item and session representations to handle popularity bias, 2019.

- [30] F Maxwell Harper and Joseph A Konstan. The movielens datasets: History and context. *Acm transactions on interactive intelligent systems (tiis)*, 5(4):1–19, 2015.
- [31] Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. Neural collaborative filtering. In *Proceedings of the 26th international conference on world wide web*, pages 173–182, 2017.
- [32] Xiangnan He, Kuan Deng, Xiang Wang, Yan Li, Yongdong Zhang, and Meng Wang. LightGCN: Simplifying and powering graph convolution network for recommendation. In *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval*, pages 639–648, 2020.
- [33] Yifan Hu, Yehuda Koren, and Chris Volinsky. Collaborative filtering for implicit feedback datasets. In *2008 Eighth IEEE International Conference on Data Mining*, pages 263–272. Ieee, 2008.
- [34] Dietmar Jannach, Lukas Lerche, Markus Zanker, and Mouzhi Ge. What recommenders recommend: an analysis of recommendation biases and possible countermeasures. *User Modeling and User-Adapted Interaction*, 25(5):427–465, 2015.
- [35] Kalervo Järvelin and Jaana Kekäläinen. Cumulated gain-based evaluation of ir techniques. *ACM Transactions on Information Systems (TOIS)*, 20(4):422–446, 2002.
- [36] Ray Jiang, Silvia Chiappa, Tor Lattimore, András György, and Pushmeet Kohli. Degenerate feedback loops in recommender systems. In *Proceedings of the 2019 AAAI/ACM Conference on AI, Ethics, and Society*, pages 383–390, 2019.
- [37] Anastasiia Klimashevskaya, Dietmar Jannach, Mehdi Elahi, and Christoph Trattner. A survey on popularity bias in recommender systems. *User Modeling and User-Adapted Interaction*, 34(4):1041–1100, 2024.
- [38] Yehuda Koren, Robert Bell, and Chris Volinsky. Matrix factorization techniques for recommender systems. *Computer*, 42(8):30–37, 2009.
- [39] Dominik Kowald and Emanuel Lacić. Popularity bias in collaborative filtering-based multimedia recommender systems. *IEEE Transactions on Multimedia*, 25:1–14, 2022.
- [40] Jürgen Leonhardt, Ashish Anand, and Megha Khosla. User fairness in recommender systems. In *Companion proceedings of the the web conference 2018*, pages 1063–1067, 2018.
- [41] Roderick J. A. Little and Donald B. Rubin. *Statistical Analysis with Missing Data*. John Wiley & Sons, Hoboken, NJ, 2 edition, 2002.
- [42] Pasquale Lops, Marco de Gemmis, and Giovanni Semeraro. Content-based recommender systems: State of the art and trends. In *Recommender systems handbook*, pages 73–105. Springer, 2011.
- [43] Majid Mansoury, Himan Abdollahpouri, Mykola Pechenizkiy, Bamshad Mobasher, and Robin Burke. Feedback loop and bias amplification in recommender systems. *Proceedings of the 29th ACM International Conference on Information & Knowledge Management*, pages 2145–2148, 2020.
- [44] Benjamin M Marlin and Richard S Zemel. Collaborative prediction and ranking with non-random missing data. In *Proceedings of the 26th annual international conference on machine learning*, pages 692–699, 2009.

- [45] Robert K. Merton. *Social Theory and Social Structure*. Free Press, New York, enlarged edition, 1968.
- [46] Eli Pariser. *The filter bubble: What the Internet is hiding from you*. Penguin UK, 2011.
- [47] Paul Resnick, Neophytos Iakovou, Mitesh Suchak, Peter Bergstrom, and John Riedl. GroupLens: an open architecture for collaborative filtering of netnews. In *Proceedings of the 1994 ACM conference on Computer supported cooperative work*, pages 175–186, 1994.
- [48] Donald B. Rubin. Inference and missing data. *Biometrika*, 63(3):581–592, 1976.
- [49] Badrul Sarwar, George Karypis, Joseph Konstan, and John Riedl. Item-based collaborative filtering recommendation algorithms. In *Proceedings of the 10th international conference on World Wide Web*, pages 285–295, 2001.
- [50] Tobias Schnabel, Adith Swaminathan, Ashudeep Singh, Navin Chandak, and Thorsten Joachims. Recommendations as treatments: Debiasing learning and evaluation. In *International conference on machine learning*, pages 1670–1679. PMLR, 2016.
- [51] Harald Steck. Training and testing of recommender systems on data missing not at random. In *Proceedings of the 16th ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 713–722, 2010.
- [52] Harald Steck. Item popularity and recommendation accuracy. In *Proceedings of the fifth ACM conference on Recommender systems*, pages 125–132, 2011.
- [53] Harald Steck. Calibrated recommendations. In *Proceedings of the 12th ACM conference on recommender systems*, pages 154–162, 2018.
- [54] Xiang Wang, Xiangnan He, Meng Wang, Fuli Feng, and Tat-Seng Chua. Neural graph collaborative filtering. In *Proceedings of the 42nd international ACM SIGIR conference on research and development in information retrieval*, pages 165–174, 2019.
- [55] Enes Yalcin and Aysen Bilge. Investigating and counteracting popularity bias in group recommendations. In *Proceedings of the 15th ACM Conference on Recommender Systems*, pages 348–357, 2021.
- [56] Fan Yao and Ke Huang. Measuring recommender system effects with simulated users. In *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency*, pages 78–88, 2021.
- [57] Zihuai Zhao, Jiahui Fan, Chen Liu, Zhefan Wen, Dawei Wang, Dawei Yin, and Jiliang Tang. Popularity bias is not always evil: Disentangling benign and harmful bias for recommendation. *IEEE Transactions on Knowledge and Data Engineering*, 35(3):2744–2757, 2022.
- [58] Ziwei Zhu, Xia Hu, and James Caverlee. Popularity bias in dynamic recommendation. *ACM Transactions on the Web (TWEB)*, 15(3):1–21, 2021.
- [59] Ziwei Zhu, Jianling Wang, and James Caverlee. Popularity-opportunity bias in collaborative filtering. In *Proceedings of the 14th ACM International Conference on Web Search and Data Mining*, pages 85–93, 2021.
- [60] Cai-Nicolas Ziegler, Sean M McNee, Joseph A Konstan, and Georg Lausen. Improving recommendation lists through topic diversification. In *Proceedings of the 14th international conference on World Wide Web*, pages 22–32, 2005.

Appendix A

Extended tables for macro and micro metrics

The following tables consolidate every model–dataset cell written by the evaluation pipeline under `Thesis_Research/outputs/results/`. Values are copied verbatim from the CSV exports (rounded for display). Re-running `run_all.py` on a different machine or seed may shift decimals slightly; the qualitative ordering of models is stable across repeated local runs.

A.1 Macro-level (catalog) metrics

CSV sources under `Thesis_Research/outputs/results/`: `combined_macro_bias_metrics.csv` and `macro_bias_metrics_take-a-lot-dataset.csv` (merged as a `take-a-lot` row). RMSE appears only where the pipeline exports a rating predictor.

Table A.1: Macro metrics (part I): concentration and popularity alignment.

Dataset	Model	Gini	ARP	Cat. cov.	$\rho_{\text{pop,rec}}$
book-crossing	ALS	0.9950	43.89	0.0111	0.1204
book-crossing	LogReg	0.4997	18.96	0.9312	0.2598
book-crossing	RandomForest	0.9678	16.54	0.2484	-0.0217
book-crossing	SVD	0.9833	31.81	0.0901	0.1826
book-crossing	TF-IDF	0.6907	20.80	0.7458	0.2096
book-crossing	k-NN	0.9360	10.87	0.2722	-0.2347
lastfm-2k	ALS	0.9928	130.63	0.0166	0.1857
lastfm-2k	LogReg	0.6829	47.15	0.9522	0.1717
lastfm-2k	RandomForest	0.8110	86.23	0.8188	0.1298
lastfm-2k	SVD	0.9712	138.93	0.1778	0.2449
lastfm-2k	TF-IDF	0.9013	132.32	0.4466	0.2212
lastfm-2k	k-NN	0.8741	12.52	0.4837	-0.3730
ml-100k	ALS	0.9874	233.09	0.0330	0.2557
ml-100k	LogReg	0.5428	75.12	0.8872	0.1318
ml-100k	RandomForest	0.9448	146.91	0.2031	0.2920
ml-100k	SVD	0.9552	157.28	0.2092	0.4339
ml-100k	TF-IDF	0.9254	61.56	0.3220	0.0739
ml-100k	k-NN	0.9800	156.04	0.0851	0.3407
ml-1m	ALS	0.9958	1178.31	0.0187	0.2015
ml-1m	LogReg	0.6102	288.67	0.9304	0.1711

Continued on next page

Table A.1 – *continued from previous page*

Dataset	Model	Gini	ARP	Cat. cov.	$\rho_{\text{pop,rec}}$
ml-1m	RandomForest	0.9829	673.02	0.0626	0.1899
ml-1m	SVD	0.9568	832.10	0.2564	0.5180
ml-1m	TF-IDF	0.9370	327.33	0.3620	0.2576
ml-1m	k-NN	0.9885	635.72	0.0709	0.2854
take-a-lot	ALS	0.9998	28.75	0.0003	0.0365
take-a-lot	LogReg	0.7332	2.22	0.5634	0.1564
take-a-lot	RandomForest	0.9952	2.77	0.0249	0.0250
take-a-lot	SVD	0.9818	11.52	0.1191	0.2908
take-a-lot	TF-IDF	0.7578	3.15	0.4896	0.2085
take-a-lot	k-NN	0.9628	5.67	0.1601	0.2236

Table A.2: Macro metrics (part II): tail mass, diversity, metadata entropy, RMSE.

Dataset	Model	Avg. pop. pct.	LT@20%	Agg. div.	Tok. ent.	RMSE
book-crossing	ALS	0.0979	0.8957	23	3.38	
book-crossing	LogReg	0.4450	0.2885	1923	7.85	
book-crossing	RandomForest	0.4887	0.2040	513	5.63	
book-crossing	SVD	0.2371	0.5696	186	6.07	1.4963
book-crossing	TF-IDF	0.3876	0.3389	1540	6.93	
book-crossing	k-NN	0.6703	0.0458	562	7.17	1.7771
lastfm-2k	ALS	0.0601	0.9999	25	8.19	
lastfm-2k	LogReg	0.4603	0.2910	1435	9.16	
lastfm-2k	RandomForest	0.3651	0.3820	1234	9.38	
lastfm-2k	SVD	0.2407	0.5389	268	9.38	0.9060
lastfm-2k	TF-IDF	0.2100	0.6471	673	9.12	
lastfm-2k	k-NN	0.7216	0.0102	729	9.17	1.3924
ml-100k	ALS	0.0693	0.9389	38	3.31	
ml-100k	LogReg	0.4669	0.2310	1022	3.65	
ml-100k	RandomForest	0.2339	0.5345	234	3.39	
ml-100k	SVD	0.1809	0.5903	241	3.38	0.9311
ml-100k	TF-IDF	0.5091	0.1556	371	2.71	
ml-100k	k-NN	0.1651	0.6420	98	3.19	1.0041
ml-1m	ALS	0.0432	0.9993	61	2.73	
ml-1m	LogReg	0.4518	0.2534	3033	3.78	
ml-1m	RandomForest	0.2582	0.5327	204	3.53	
ml-1m	SVD	0.1923	0.6882	836	3.46	0.8726
ml-1m	TF-IDF	0.4058	0.2869	1180	2.86	
ml-1m	k-NN	0.2244	0.6156	231	3.53	0.9769
take-a-lot	ALS	0.0026	1.0000	14	6.09	
take-a-lot	LogReg	0.4633	0.2028	27556	10.01	
take-a-lot	RandomForest	0.4110	0.2890	1217	8.44	
take-a-lot	SVD	0.1015	0.8215	5824	9.34	0.9662
take-a-lot	TF-IDF	0.3940	0.3045	23945	10.07	
take-a-lot	k-NN	0.2251	0.5749	7830	9.61	1.0211

A.2 Micro-level (user-centric) metrics

Segment metrics mirror `combined_user_centric_metrics.csv` and `user_centric_metrics_take-a-lot-dataset.csv`. ΔRMSE is niche minus mainstream RMSE (positive implies higher error on niche users).

Table A.3: Micro metrics (part I): RMSE by taste segment and ΔRMSE .

Dataset	Model	$\text{RMSE}_{\text{niche}}$	$\text{RMSE}_{\text{div.}}$	$\text{RMSE}_{\text{main}}$	ΔRMSE
book-crossing	ALS	1.7427	1.6719	1.7295	0.0132
book-crossing	LogReg	1.7427	1.6719	1.7295	0.0132
book-crossing	RandomForest	1.5814	1.5817	1.6347	-0.0533
book-crossing	SVD	1.4845	1.4927	1.5165	-0.0320
book-crossing	TF-IDF	1.7427	1.6719	1.7295	0.0132
book-crossing	k-NN	1.8546	1.7226	1.7419	0.1127
lastfm-2k	ALS	1.3925	1.4177	1.4172	-0.0247
lastfm-2k	LogReg	1.3925	1.4177	1.4172	-0.0247
lastfm-2k	RandomForest	1.1766	1.2507	1.2454	-0.0688
lastfm-2k	SVD	0.8364	0.9051	0.9503	-0.1140
lastfm-2k	TF-IDF	1.3925	1.4177	1.4172	-0.0247
lastfm-2k	k-NN	1.3947	1.4217	1.3643	0.0304
ml-100k	ALS	1.1354	1.0948	1.0721	0.0633
ml-100k	LogReg	1.1354	1.0948	1.0721	0.0633
ml-100k	RandomForest	1.0442	1.0221	1.0201	0.0241
ml-100k	SVD	0.9346	0.9253	0.9294	0.0052
ml-100k	TF-IDF	1.1354	1.0948	1.0721	0.0633
ml-100k	k-NN	1.0235	0.9802	0.9755	0.0480
ml-1m	ALS	1.1471	1.0800	1.0748	0.0723
ml-1m	LogReg	1.1471	1.0800	1.0748	0.0723
ml-1m	RandomForest	1.0191	0.9696	0.9671	0.0520
ml-1m	SVD	0.8740	0.8677	0.8772	-0.0032
ml-1m	TF-IDF	1.1471	1.0800	1.0748	0.0723
ml-1m	k-NN	0.9975	0.9482	0.9493	0.0482
take-a-lot	ALS	1.0235	1.0655	0.9467	0.0769
take-a-lot	LogReg	1.0235	1.0655	0.9467	0.0769
take-a-lot	RandomForest	1.0229	1.0610	0.9409	0.0819
take-a-lot	SVD	0.9663	1.0218	0.9059	0.0604
take-a-lot	TF-IDF	1.0235	1.0655	0.9467	0.0769
take-a-lot	k-NN	1.0249	1.0763	0.9572	0.0677

Table A.4: Micro metrics (part II): NDCG@10 by taste segment.

Dataset	Model	$\text{NDCG}_{\text{niche}}$	$\text{NDCG}_{\text{div.}}$	$\text{NDCG}_{\text{main}}$
book-crossing	ALS	0.0124	0.0123	0.0116
book-crossing	LogReg	0.0592	0.0502	0.0408
book-crossing	RandomForest	0.0095	0.0076	0.0085
book-crossing	SVD	0.0024	0.0085	0.0101
book-crossing	TF-IDF	0.1120	0.0821	0.0737
book-crossing	k-NN	0.0008	0.0016	0.0029

Continued on next page

Table A.4 – *continued from previous page*

Dataset	Model	NDCG_{niche}	NDCG_{div.}	NDCG_{main}
lastfm-2k	ALS	0.0210	0.0509	0.0254
lastfm-2k	LogReg	0.0310	0.0284	0.0743
lastfm-2k	RandomForest	0.0132	0.0155	0.0642
lastfm-2k	SVD	0.0157	0.0215	0.0914
lastfm-2k	TF-IDF	0.1507	0.1403	0.2557
lastfm-2k	k-NN	0.0000	0.0008	0.0001
ml-100k	ALS	0.1709	0.1004	0.0727
ml-100k	LogReg	0.0653	0.0305	0.0151
ml-100k	RandomForest	0.1044	0.0644	0.0363
ml-100k	SVD	0.1465	0.0805	0.0435
ml-100k	TF-IDF	0.0660	0.0346	0.0234
ml-100k	k-NN	0.1514	0.0828	0.0416
ml-1m	ALS	0.1396	0.0902	0.0508
ml-1m	LogReg	0.0396	0.0193	0.0120
ml-1m	RandomForest	0.0604	0.0395	0.0251
ml-1m	SVD	0.1274	0.0875	0.0572
ml-1m	TF-IDF	0.0697	0.0549	0.0327
ml-1m	k-NN	0.0792	0.0434	0.0231
take-a-lot	ALS	0.0002	0.0010	0.0017
take-a-lot	LogReg	0.0038	0.0064	0.0043
take-a-lot	RandomForest	0.0000	0.0000	0.0004
take-a-lot	SVD	0.0001	0.0001	0.0015
take-a-lot	TF-IDF	0.0117	0.0118	0.0175
take-a-lot	k-NN	0.0013	0.0011	0.0004

A.3 Longitudinal feedback simulation (full iteration grid)

All iterations for SVD and TF-IDF from `combined_feedback_simulation_metrics.csv`. The main text stresses qualitative trajectories; this grid supports exact reproduction of line charts.

Table A.5: Feedback simulation: aggregate diversity and ARP at every iteration (SVD vs. TF-IDF).

Dataset	Model	Iter.	Agg. diversity	ARP
book-crossing	SVD	1	134	32.79
book-crossing	SVD	2	306	26.26
book-crossing	SVD	3	533	24.34
book-crossing	SVD	4	609	24.50
book-crossing	SVD	5	715	23.63
book-crossing	TF-IDF	1	1264	21.29
book-crossing	TF-IDF	2	1330	24.78
book-crossing	TF-IDF	3	1348	27.52
book-crossing	TF-IDF	4	1382	30.13
book-crossing	TF-IDF	5	1404	32.16
lastfm-2k	SVD	1	170	176.66
lastfm-2k	SVD	2	103	245.06
lastfm-2k	SVD	3	129	560.07

Continued on next page

Table A.5 – *continued from previous page*

Dataset	Model	Iter.	Agg. diversity	ARP
lastfm-2k	SVD	4	148	661.76
lastfm-2k	SVD	5	166	699.74
lastfm-2k	TF-IDF	1	492	150.24
lastfm-2k	TF-IDF	2	516	174.06
lastfm-2k	TF-IDF	3	532	188.95
lastfm-2k	TF-IDF	4	531	197.58
lastfm-2k	TF-IDF	5	538	202.75
ml-100k	SVD	1	174	165.14
ml-100k	SVD	2	141	264.26
ml-100k	SVD	3	157	335.54
ml-100k	SVD	4	143	358.38
ml-100k	SVD	5	162	405.70
ml-100k	TF-IDF	1	235	68.34
ml-100k	TF-IDF	2	286	89.03
ml-100k	TF-IDF	3	333	101.94
ml-100k	TF-IDF	4	366	109.40
ml-100k	TF-IDF	5	394	115.63
ml-1m	SVD	1	649	893.35
ml-1m	SVD	2	541	918.07
ml-1m	SVD	3	523	1321.16
ml-1m	SVD	4	528	1832.50
ml-1m	SVD	5	529	1997.48
ml-1m	TF-IDF	1	805	348.38
ml-1m	TF-IDF	2	983	494.43
ml-1m	TF-IDF	3	1084	556.39
ml-1m	TF-IDF	4	1183	576.73
ml-1m	TF-IDF	5	1278	584.20

Appendix B

Additional visualizations and diagrams

This appendix collects schematic views of the evaluation workflow and reproduces the cross-dataset figures from the main experiments at appendix scale, so printed copies remain self-contained even when individual chapters are excerpted.

B.1 End-to-end evaluation pipeline

Figure B.1 situates Experiments A–C as three measurement stations on a shared data and training backbone: the same splits, models, and recommendation lists feed macro catalog metrics, segment-level user metrics, and the longitudinal click simulator.

B.2 Stylised feedback loop with explicit simulator knobs

Figure B.2 summarises the feedback simulation at the level of state variables referenced in Chapter 6. Clicks use the linear blend $c_{ui} = \text{clip}(0.7 s_{ui} + 0.3 \tilde{p}_i, [0, 1])$ from Section 3.9, then a Bernoulli draw on each recommended slot.

B.3 Cross-dataset figures (same assets as the main text)

For convenience, the heatmaps and segment plots used in Chapters 4–6 are repeated here without reinterpretation.

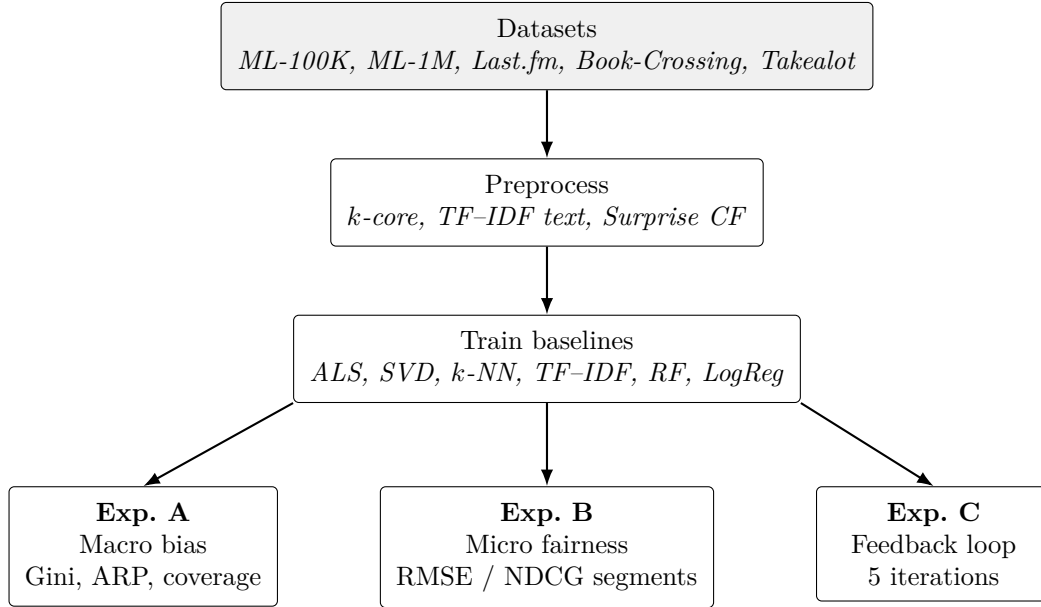


Figure B.1: Unified pipeline: shared preprocessing and model training feed three evaluation tracks (Chapters 4–6).

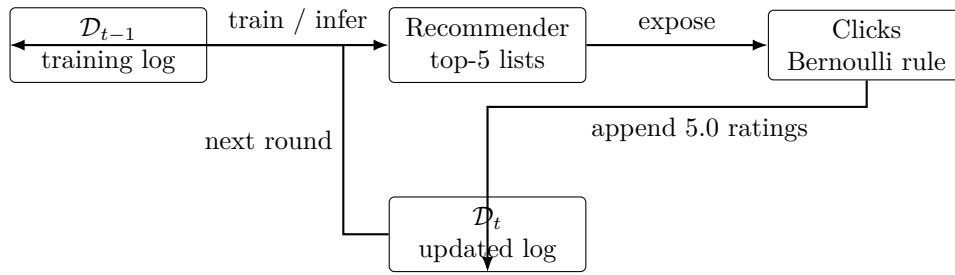


Figure B.2: Longitudinal simulator: logs evolve from recommendations and stochastic clicks, then re-enter training.

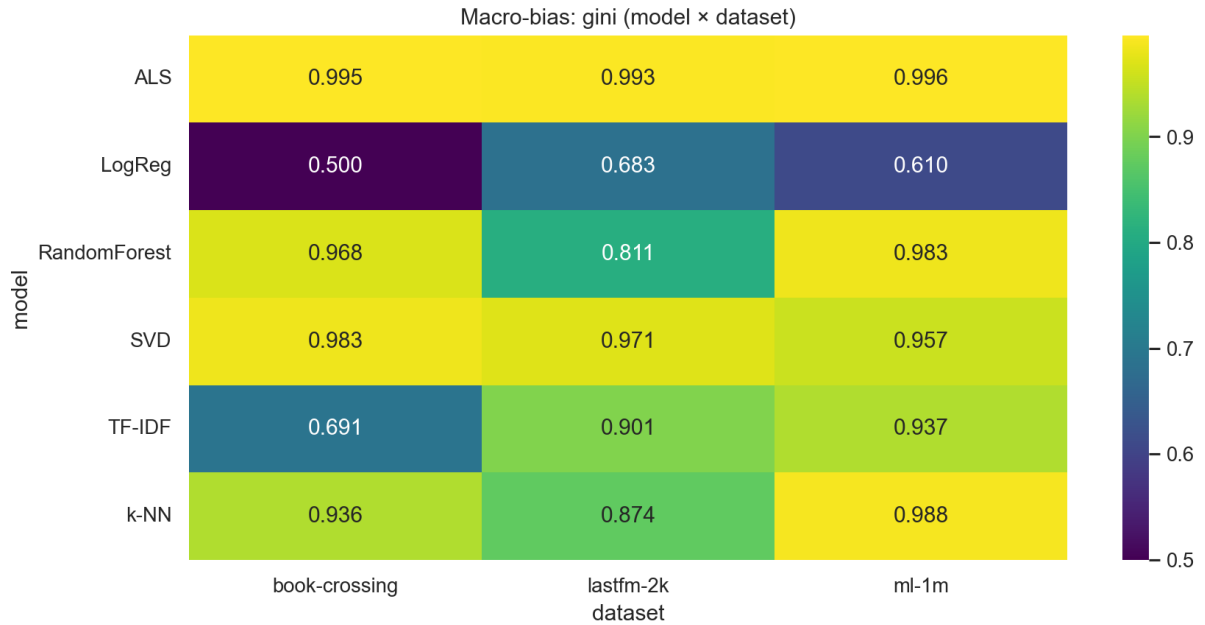


Figure B.3: Gini coefficient of recommendation frequency versus item popularity (all non-Takealot benchmarks).

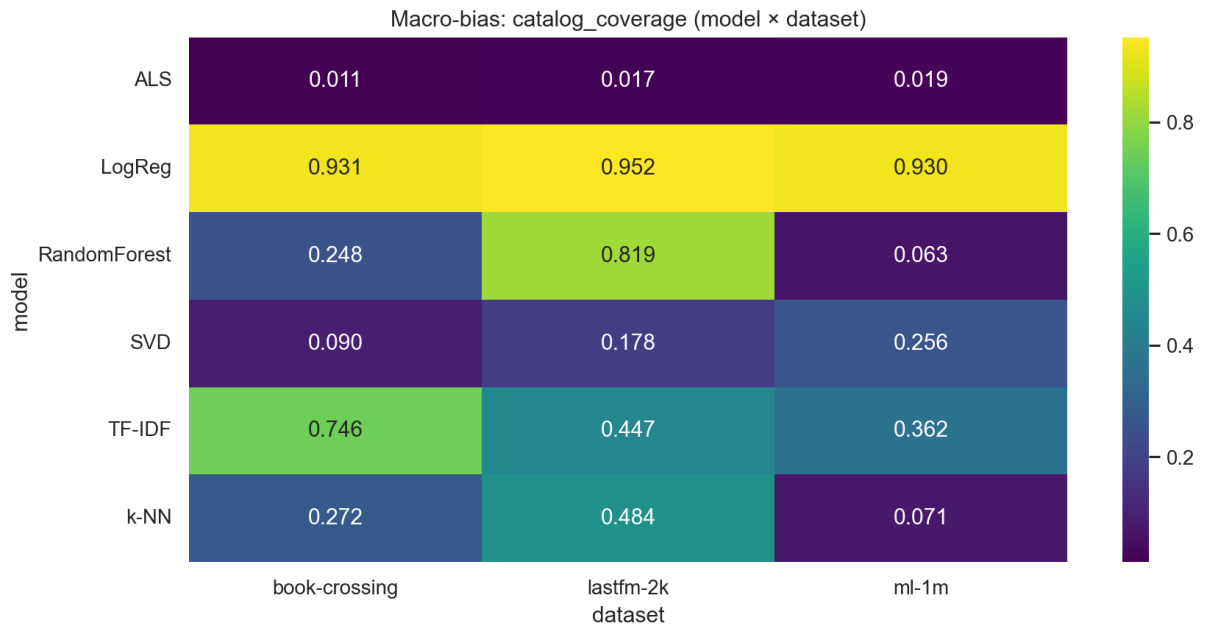


Figure B.4: Catalog coverage of top- N recommendations across datasets and models.

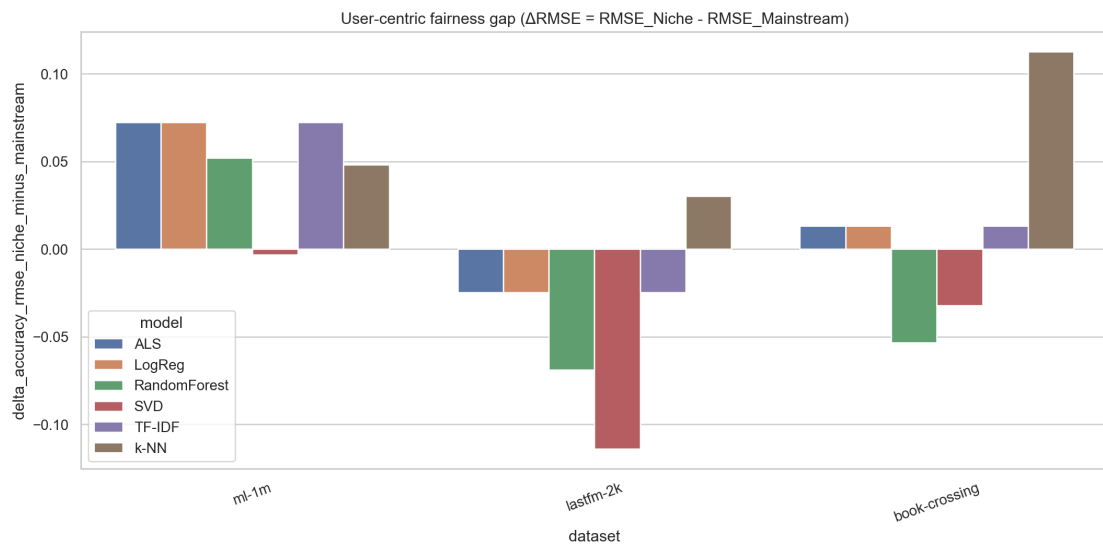


Figure B.5: Niche minus mainstream RMSE (ΔRMSE) by dataset and model.

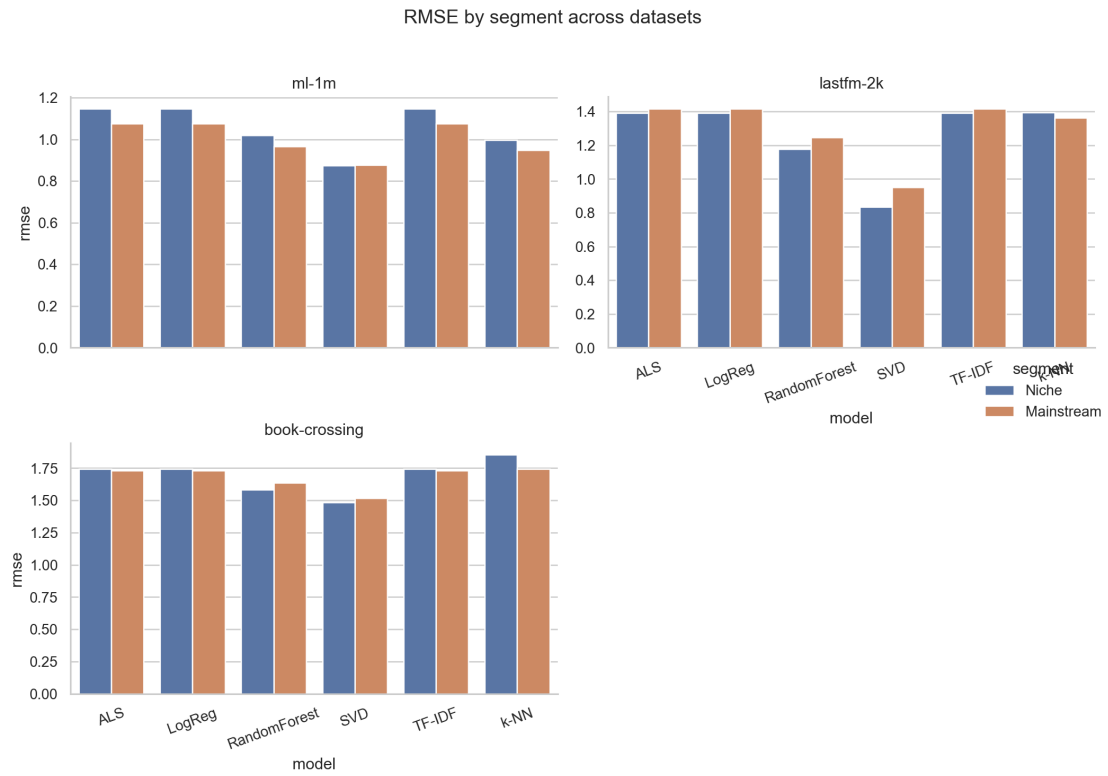


Figure B.6: RMSE for niche versus mainstream taste segments.

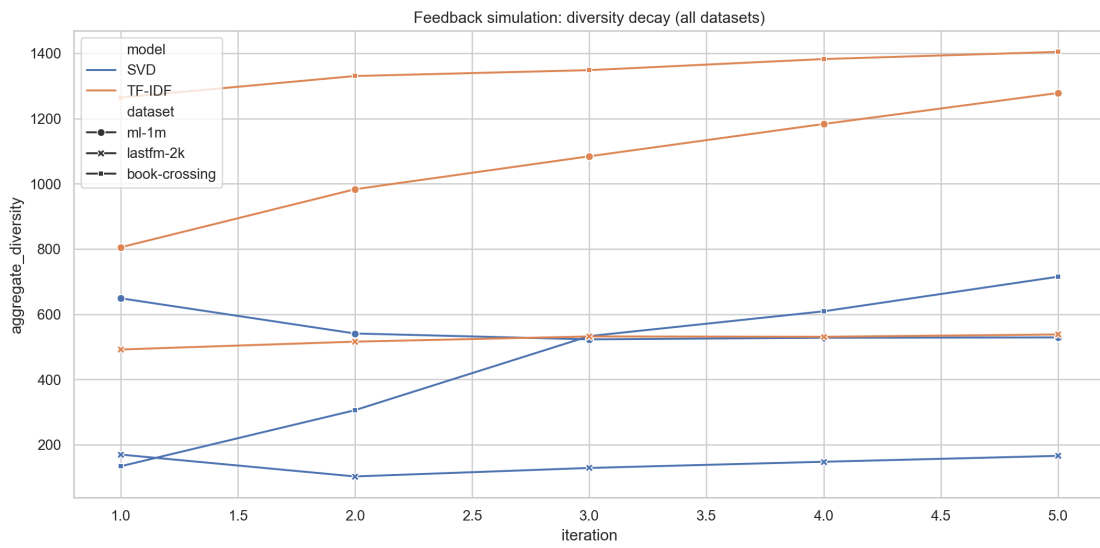


Figure B.7: Aggregate diversity and ARP across five feedback iterations (SVD versus TF-IDF).

Appendix C

Extended software specifications

C.1 Repository layout

All experiments are implemented as a single Python package-style tree under `Thesis_Research/`: `run_all.py` orchestrates dataset ingestion, training, evaluation, plotting, and CSV exports; `src/` holds metric utilities and statistical tests; `outputs/results/` stores the tables consumed by this document. Figures are written to `outputs/figures/` (and mirrored under `Thesis-main/outputs/figures/` when syncing for L^AT_EX).

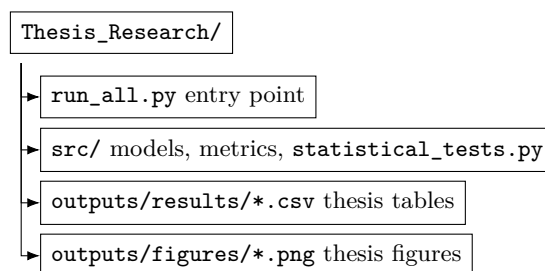


Figure C.1: Code and artefact layout relative to the research repository root.

C.2 Language runtime and hardware assumptions

The code targets **Python 3.10+** (tested on 64-bit Windows 10/11). Experiments are CPU-bound; GPU acceleration is not required. Wall-clock runtime scales with Book-Crossing and Takealot matrix sizes; a full multi-dataset sweep completes overnight on a modern laptop when Surprise and implicit builds are already compiled.

C.3 Core dependency pins

Table C.1 lists the libraries that materially affect numerical results. A broader scientific stack (Jupyter, widgets, conda solver libraries) may be present in a full Anaconda install but does not change the reported metrics.

C.4 Pinned versions (pip freeze excerpt)

Listing C.1 lists pinned versions from the development host. Reproduce with a clean virtual environment and the research repository `requirements.txt`, then align versions as needed.

Package	Role in this thesis	Notes
numpy	Arrays, linear algebra	Pinned below 2.x for <code>implicit</code> compatibility.
pandas	CSV aggregation	Used for all combined result tables.
scipy	Statistics	Welch/Mann–Whitney helpers in <code>statistical_tests.py</code> .
scikit-learn	TF-IDF, RF, LogReg, metrics	Pipelines mirror standard sklearn idioms.
scikit-surprise	SVD, k -NN, ALS	Optional import guarded when wheels are unavailable.
implicit	ALS (implicit feedback path)	Used where the benchmark supports implicit-style training.
matplotlib, seaborn	Plots	Heatmaps and line charts exported to PNG.
tqdm	Progress bars	Long Takealot sweeps remain observable during batch runs.

Table C.1: Scientific stack directly exercised by `run_all.py`.

Listing C.1: Excerpted `pip` pins for core scientific dependencies.

```
# Core Python packages pinned on the host used for thesis runs (May 2026).
# Regenerate: pip install -r Thesis_Research/requirements.txt && pip freeze | findstr /i "numpy
  pandas scipy scikit surprise implicit matplotlib seaborn tqdm notebook ipywidgets"
implicit==0.7.2
matplotlib==3.10.8
numpy==1.26.4
pandas==3.0.1
scikit-learn==1.8.0
scikit-surprise==1.1.4
scipy==1.17.1
seaborn==0.13.2
tqdm==4.67.3
ipywidgets==8.1.8
notebook==7.5.5
```

C.5 Regeneration checklist

1. Install dependencies from `requirements.txt` in the research repo; verify `surprise` imports.
2. From the repo root, run `python run_all.py` with the thesis defaults (top-10 lists for offline metrics; feedback uses top-5 and five iterations).
3. Copy new PNGs into folder `Thesis-main/outputs/figures/` when compiling the thesis locally.
4. Run `python scripts/export_appendix_tables.py` to rebuild `_appendix_generated_tables.tex`.
5. Recompile `thesis.tex` (`pdflatex`, `bibtex`, `pdflatex` $\times 2$).